MasSecEval

(Multi Agent System Security Evaluation)

By

Muhammad Usman Baig

Shehriar Ali Khan

Muhammad Fazal

Shah Nawaz Ali Asghar

Supervised by:

Dr. Samra Kanwal

Submitted to the faculty of Department of Computer Software Engineering,

Military College of Signals, National University of Sciences and Technology,
Islamabad,
in partial fulfillment for the requirements of B.E Degree in Software Engineering.

April 2026

In the name of ALLAH, the Most Benevolent, the Most Courteous

CERTIFICATE OF CORRECTNESS AND APPROVAL

This is to officially state that the thesis work contained in this report

"MasSecEval"

is carried out by

Muhammad Usman Baig

Shehriar Ali Khan

Muhammad Fazal

Shah Nawaz Ali Asghar

Under my supervision and that in my judgement, it is fully ample, in scope and excellence, for the degree of Bachelor of Software Engineering in Military College of Signals, National University of Sciences and Technology (NUST), Islamabad.

Approved by

Supervisor

Dr. Samra Kanwal

Date:_____

DECLARATION OF ORIGINALITY

We hereby declare that no portion of work presented in this thesis has been submitted in support of another award or qualification in either this institute or anywhere else.

ACKNOWLEDGEMENTS

We thank our Almighty Allah Subhana Watalah who has blessed and guided us through our final year project, which is software engineering. His constant support and guidance could not have been valued enough without which we would not have been where we are today.

Our parents have always been our main motivator and encouragement and we owe them a lot of credit to their unconditional encouragement and support. Their sacrifice and commitment to our education is what has formed our success.

It is our pleasure to recognize that our supervisor, Dr. Smara Kanwal, has been incredibly helpful in this project since we asked her to mentor and guide us. Her professional skills, tolerance, and helpful comments were very important in the development of our thoughts and improvement of our work.

We also wish to thank the faculty and staff of the Computer Software Engineering department who helped and trained us during our course work. Their advice and expertise have played a central role in our perception of the topic of study and how this project has been accomplished.

In addition, we would like to express our utmost gratitude to the worldwide open source cybersecurity circles and their contributors keeping the OpenTelemetry and OWASP structures with their indispensable group-wide norms. Particularly, we owe much of our success to the engineering teams that created the generative language models that we used; their ample documentation and access to a solid API was essential to constructing the deterministic auditing pipelines in MasSecEval. Such an essential input of open diagnostic standards and accessibility of technology played a key role in the success of the project and its possible practical applicability to the software industry.

Lastly, we would like to mention to our friends, colleagues, and well wishers that we lose our lives in this process because we are not alone in doing so.

PLAGIARISM CERTIFICATE (TURNITIN REPORT)

This thesis has 10% similarity index. Turnitin report endorsed by Supervisor is attached.

Muhammad Usman Baig

NUST Serial no: _____

Shah Nawaz Ali Asghar

NUST Serial no: _____

Shehriar Ali Khan

NUST Serial no: _____

Muhammad Fazal

NUST Serial no: _____

_____

Signature of Supervisor

ABSTRACT

The Static Application Security Testing (SAST) is a fundamental concept in the contemporary software development environment, as it offers a methodological look at the security stance of a codebase and its possible weaknesses. Nevertheless, the manual generation of security scan results is usually a tedious and lengthy exercise and most vulnerable to diagnostic exhaustion, excessive number of false positives and the vital inability of the code to identify any theoretical weaknesses and truly viable code pathways. Although classic security solutions have demonstrated potential in detecting simple syntactical anomalies, the current software utilized in the Devops lifecycle can be seen as centered around individual, rule based activities such as library testing or finding a small number of, deprecated functions. More importantly, only a few integrated systems exist, which can not only detect security pathologies by means of sophisticated artificial intelligence, but also be able to map them to live execution traces to helpful, verified developer insights. Moreover, most specialised security software is characterised by complicated, unfriendly interfaces and high learning curves and therefore cannot be easily integrated into the day-to-day processes of developers.

To mitigate such drawbacks, we present MasSecEval, AI-based security evaluation platform, which is aimed at automatizing the hybrid analysis of source code and can act as a highly effective diagnostic support system of the software professional. MasSecEval is unique in its twofold design: by utilizing the power of the state of the art generative models to identify and localize vulnerabilities (accurately) and a user friendly interface that evolved under a stricter Human Computer Interaction (HCI) design guidelines. It is proposed to inoculate way beyond traditional security suites in terms of usability with multi language support (English and Urdu), guided onboarding, and accessibility options in terms of multiple theme selection and font sizes to enable one to operate with ease in cases of complex security audit.

MasSecEval consists of a complex pipeline based on the microservices architecture deployed in the FastAPI and Next.js frameworks. The system is hosted on a distributed backend server and uploaded codebases are analyzed in a multi stage AI analysis: a deterministic model would identify the vulnerabilities using the OWASP Top 10 guidelines, and a secondary Retrieval Augmented Generation (RAG) system

would offer contextualized remediation guidelines. At the same time, an OpenTelemetry (OTEL) receiver will capture operational traces of applications in real-time to match detected states with their respective execution paths. The system is based on the high confidence auditing of the Google Gemini AI model and is supported by a custom vectorized dataset that is stored using the Qdrant semantic search and developer assistance engine. The user communicates with the platform through a responsive React frontend that exchanged information with the backend over Active task tracking protocols in both forms of RESTful API and WebSocket.

Among the objectives of the successful development of the MasSecEval, it is planned to provide developers with a powerful tool, which simplifies the phenomenon of the vulnerability interpretation and presents the rapid, consistent and precisely verified data of the diagnostic diagnosis in the form of the intuitive one. MasSecEval can optimize the security efficiency and facilitate critical decision making process in the software development life cycle by enabling quicker triage, providing more insightful results of automated, static and dynamic maps. The focus on higher usability and accessibility should help integrate even more easily into professional clinical or corporate settings and eventually increase the resilience and security of the software infrastructure.

CONTENTS

LIST OF FIGURES

CHAPTER 1 INTRODUCTION

The chapter presents the Multi Agent System Security Evaluator, the multi-agent system of software security diagnostic assistance, which employs the framework of artificial intelligence. It sketches the problem statement that the project will deal with, the proposed solution, the aims and objectives, the project scope, the working principle, the project deliverables list, and the discussion of the sustainable development goals that are relevant.

1.1 Overview

Software security is an essential aspect of the stability of the whole digital infrastructure, and proper vulnerability diagnosis is the key to the successful threat prevention. Listening to the range of diagnostic instruments which cybersecurity professionals can use, Static Application Security Testing is one of them that has a certain importance. A single analysis can show the whole application architecture, backend logic, database queries and frontend routing allowing a static code scanner to be valuable as a general security screening tool, compliance assessment tool, and architectural planning tool. Classically, these complicated interactions between source code are interpreted solely by the visual perception and experience of senior security engineers.

In recent years technology more specifically the concept of generative Artificial Intelligence and Large Language Models has made an enormous impact on many areas that include software engineering. Artificially intelligent code analysis has proven to have enormous potential in increasing the level of diagnostic accuracy, uniformity and efficiency in different development structures. In application security semantic understanding models came in with sensational potential in activities like data flow contextualization and highlevel logic errors. This development represents a huge advance to augmenting the traditional diagnostic security workflows by the use of automated tools of generative analysis.

In the realization of the potential of the use of generative technology in this area the MasSecEval project will produce an integrated platform that uses large language models to automatically analyze the source code. It does not just aim at identifying certain software vulnerabilities but also precisely mapping these

hypothetical results to live execution traces. One of the fundamental concepts of MasSecEval is that it combines advanced analytical pipeline engines with highly user centric user interface evolution provide based on more modern principles of interaction to guarantee ease of use, accessibility and controlled integration into development processes unlike more traditional specialized command line software that frequently requires special supporting technical training.

1.2 Problem Statement

Although in a demonstration of diagnostic information, the static security scanners are rich in information, its interpretation is done manually, with some challenges associated with it. The auditing may be excruciatingly time consuming especially when dealing with complex enterprise codebases or high velocity deployment environments. Moreover, visual interpretation by human listeners is subjective in nature and this may result in inconsistent security postures among teams of seasoned listeners. There is also possibility that diagnostic fatigue may also have a serious consequence on accuracy when a long review session is involved. A deep level of attention should be paid to identification of subtle logic mistakes and monitoring the process of data sanitization over time; it was necessary to have specific experience.

Even though the literature on cybersecurity has already addressed the problem of automation of the vulnerability detection, numerous massive gaps still exist. A lot of work that has been done is based on single actions e.g. searching deprecated libraries back using bare pattern matching without knowledge about the surrounding execution environment. Those studies which do tackle complex disease detection in the code normally span over a narrow scope of structures or omit the critical step of overlaying the detected pathology to definite dynamic runtime trace. This mapping is critical data in the clinical documentation and real-life treatment planning by developers. Commercial solutions to such services do exist, but they are mostly proprietary, and expensive to subscribe to, and cannot match the level of open research transparency, which significantly restricts accessibility and independent validation.

Another important, and in many cases totally underestimated, issue is the usability of specialized diagnostic software. Most of the available security tools have

complicated interface, which may be hard to navigate and may need considerable user training, which can be an impediment to use, particularly by the less knowledgeable junior developers who may not have adequate background with high end security concepts. The current necessity is obvious to have tools that are more than just technically advanced but also intuition-oriented and include such features as a contextual chat support, an accessibility option, and up-to-date interactive paradigms. Absence of cohesive, precise, readily available, and consumer friendly intelligence system to undertake thorough security analysis is a major unfulfilled requirement with the current software development life cycle.

1.3 Proposed Solution

To solve the specified problems in manual software code interpretation and usability, we suggest MasSecEval an artificial intelligence based online application that will be an effective diagnostic aid device in software developers. The MasSecEval is aimed at automated identification of common security conditions and localization of them through the source code with the findings that are visualized in a highly intuitive interface taking into consideration modern user experience principles. Such usability features as the support of natural language queries and integrated Retrieval Augmented Generation chatbot and transparent visual data mapping are oriented to the easy introduction into everyday engineering activities.

Project code uploads demand from the system are processed via a distributed backend pipeline, which is run on a FastAPI server. It makes use of particular analytical services one addresses deterministic auditing on the basis of the Google Gemini model on OWASP vulnerabilities the other handles ingestion and embedding of code in a Qdrant vector database and the third one intercepts live OpenTelemetry traffic on deployed applications. One of these fundamental innovations is the following mapping algorithm that adds the results of these static and dynamic product to a perfect match between the identified vulnerabilities and individual execution spans. The last organized diagnostic data is returned thenceforth to the user friendly Next.js frontend supplying the skilled programmer with speedy, localized and an easily comprehensible remedial information to assist in their security evaluation.

1.4 Working Principle

The MasSecEval project is based on the concept of an advanced generative evaluation, that is, it uses state of the art language models to analyze application health as well as live telemetry parsers. It is designed in the form of a client server web application, whereby, user interaction follows different stages that are followed by the backend processing by an asynchronous worker pipeline, and presentation of results by a user centric graphical interface.

1.4.1 Backend AI Pipeline

When a project repository is uploaded by the user through the frontend interface, the FastAPI backend triggers a multi stage processing pipeline. This pipeline is used to perform three separate operation phases sequentially relying on Celery workers and a Redis message broker.

The Static Analysis Model will work with the original source code that is stored in MinIO and will help to reveal and define particular logic errors and syntax flaws by working solely with the parameters of the OWASP set of the top 10.

Contextual Embedding Model: The Contextual Embedding Model indexes the whole source code to compute semantic vectors, which are stored in a Qdrant database to drive the retrieval augmented generation engine.

Dynamic Telemetry Model works simultaneously to hear OpenTelemetry HTTP and gRPC protocols, taking on the active execution traces of the deployed target application.

The outputs were combined using a critical mapping algorithm, after the collection stage. It consists of the computations of the functional overlap between the theoretically observed static vulnerabilities and the active span names and status codes observed by the telemetry receiver. Any identified vulnerability instance is thus properly attributed to the particular network request that it impacts, developing a detailed, localized diagnostic image.

1.4.2 Model Architecture Approach

In contrast to normal pattern matching or simple syntax tree parsing algorithms, MasSecEval uses a generative semantic learning method. Gemini model integration was selected because it is an effective pattern of complex reasoning tasks and is able to detect, classify, and generate remedies that are highly specific to various

security defects in the source code simultaneously. Separated worker queues with each worker queue being picked by its task such as embedding generation or trace parsing can be processed with focus and may be more accurate than a single all-purpose function trying to do all the tasks at the same time. These models compare code with the available industry data and guidelines where ground truth security standards provide the guiding analytical process.

1.4.3 GUI Presentation

The front of visually interacting and presenting MasSecEval is managed by a Graphical User Interface built upon Next.js framework. Following contemporary design rules, the front-end offers the experience of easy uploading the project files, connecting repositories, or seeing diagnostic outcomes. The application gets the processed data, which is sent to it by the FastAPI backend API in the form of a JSON file and interprets this information and presents it dynamically. The outcomes are usually displayed in a form of a well-organized table that views the vulnerabilities identified and their respective code snippets and dynamic traces evidence. Also, the interactive capabilities provide the user with the ability to communicate directly to the intelligent chatbot about their particular codebase. A wide range of components and libraries are integrated into the interface to provide such features as real time status update, customization of dark mode, and clear visual hierarchies to make it readily accessible and to ensure its usability by development professionals.

1.5 Aims & Objectives

1.5.1 General Objectives

- Finding a way to develop a more sophisticated system that would act as an aid to software engineers to automate large portions of static and dynamic security analysis.
- Also, it will advance the effectiveness and consistency of detecting and locating typical application weaknesses, which can lead to greater diagnostic precision throughout the development cycle.
- Developing a support diagnostic tool that is useful and accessible in the corporate environment with a great focus on the user experience that will be well-designed on the principles of modern interfaces.

- Helped to equip and enhance generative artificial intelligence to the domain of software security auditing to have a practical implementation.

1.5.2 Academic Objectives

- **Marketing:** To develop, implement, and test a complex microservices architecture that is able to carry out accurate deterministic static analysis of current web frameworks.
- To develop and test a new processing pipeline, successfully combining the use of both OpenTelemetry runtime metrics with a stateful set of code intelligence, to translate a breach of theory to an active dynamic endpoint.
- The preparation of a complete web application, which includes a convenient frontend with the usage of particular interactive components, such as a chatbot focused on a conversational style and a live update, supported by a powerful API in order to manage the entire process and have a pipeline running.
- **The proposed methodology:** To mechanically analyze the quantitative effectiveness of the retrieval augmented generation models as measured by the applicable contextual measures on the relevant test repositories.
- To use reasonable software engineering practices throughout the project lifecycle and generate extensive documentation of the system design, deployment plan and technical implementation process.

1.6 Scope

The MasSecEval project is mainly aimed at automating the main points of the security analysis to aid in software diagnostics. The system is created to receive digital team source code file archives or repository links as an input. Its central functionality contains automated vulnerability scanning of codebase, vegetation of content into semantic search functionality and identifying whether certain flaws belong to the OWASP Top 10 methodology. One of the areas is the used pipeline that tracks the detected static conditions to the mapped OpenTelemetry traces accurately.

The system provides its analysis through best web application with user focus interface with particular upgrades such as built-in security assistant. The requirements of this functionality comprise the implementation of the frontend client and backend orchestration elements.

This project does not seek to provide the service of a live Web Application Firewall or an active intrusion prevention system. The identification features mostly center on logic and syntax errors; active manipulation or reverse engineering of network packets are beyond the current view of detection. MasSecEval is a diagnostic assistance tool, and does not offer automated commits made to production servers. This stage of development does not include integration with external issue tracking software that is currently in place.

1.7 Deliverables

1.7.1 Fine Tuned Analytical Pipeline

An analytical engine, implemented as a distributed system written in Python, Celery, Redis, using task queues. It is a backend that is specifically tuned to:

- Generative prompting static code auditing.
- Retrieval Augmented Generation context building.
- Real-time telemetry theft and information correlation.

Containerized deployment scripts and environment configuration files are key deliverables.

1.7.2 Web Application

An operational web-based application that has:

- **Frontend:** Built with Next.js and the tailwind CSS library, which was used to build the user interface, logic of interaction, RESTful communication and data representation.
- **Backend:** Built in FastAPI, and contains the API endpoints, manages database interactions with PostgreSQL, and runs the detection and mapping pipeline.

1.7.3 User Manual

A full technical guide on how the administrators can apply the infrastructure through Docker Compose and how the users can use the platform.

1.7.4 Short Demo Video

A short video showing the most important functionalities, project ingestion workflow and dialogues using conversational chatbots of the MasSecEval application.

1.8 Relevant Sustainable Development Goals

Sustainable Development Goals may be defined as the global development agenda to eliminate extreme poverty, curb inequality and preserve the environment by the year 2030. Among the global objectives, the architecture of MasSecEval is mostly traced on two objectives. Our project is mapped on the first objective which is:

Goal 9 — Industry, Innovation and Infrastructure:

This project reflects profound technical innovation in that it implements state of the art artificial intelligence and distributed telemetry algorithm to a particular problem in the software industry. The creation of advanced software-based infrastructure such as MasSecEval, which combines advanced machine learning models and an easy to use interface, promotes the state of technology and creates incredibly robust digital infrastructure in the world of technology industry.

SDG 16 — Peace, Justice and Strong Institutions:

MasSecEval promotes the safety of confidential information and cyber exploitation prevention by seeking considerably to enhance the efficiency and consistency of vulnerability diagnostics. Quick and more convenient diagnostic assistance can result in earlier discovery of vital vulnerabilities, and eventually ensure enhanced security of institutions and protection of digital identity against unscrupulous users.

1.9 Structure of Thesis

Chapter 2: Literature Review contains the industrial background, existing security solutions, and their specific drawbacks.

Chapter 3: Methodology contains the software tools, development techniques, and architectural approaches.

Chapter 4: Software Requirements Specifications

Chapter 5: Detailed Design and Architecture

Chapter 6: Conclusion and Future Work highlights the future work needed to be done for the commercialization of this project.

Chapter 7: Plagiarism Report is a plagiarism report of this thesis document.

CHAPTER 2 LITERATURE REVIEW

To create something new such as MasSecEval, one needs to be fully aware of the current state of the situation. Research on similar work and related studies is an important process that would enable determination of gaps, limitations, and locating the contribution of a new project. The chapter is a review of existing literature concerning Artificial Intelligence in application security in the context of automated analysis of source code and active telemetry.

The framework of our review is the following:

- **Industrial Background:** Discusses the big picture and the potential of Artificial Intelligence in software diagnostics.
- **Existing Solutions and Flaws:** Investigates particular previous research, application packages, and test pipelines, their drawbacks.
- **Proposed Project:** Justifies the MasSecEval project by describing how it will cover the gaps developed.

2.1 Industrial Background

Generative deep learning and the Artificial Intelligence are quickly changing other parts of software engineering, application security being one of the most affected areas. Intelligence driven systems are now being variously used to help developers in diverse deployment environments, with researchers using them to perform code repositories vulnerability analysis, including access control vulnerabilities and injection vulnerabilities, to achieve an efficient deployment with reduced security risks.

These developments are gradually finding application in application security though this may not be adopted as faster than the general code generation. The generative analysis is particularly appropriate with Static Application Security testing which provides a broad panoramic perspective of the internal software architecture including not only logic but also database queries and other supporting infrastructure as well. The range of possible uses is wide and includes automated syntax assessment data flow representation recognition of logic anomalies, and even allowing architectural refactoring. These complicated repositories are manually interpreted with high levels of training and experience required to interpret data sets and improve

oversight and diagnostic fatigue although prone to diagnostic fatigue. The limited volume of security engineers in most organizations and those with a high release cycle is such that attempting to treat security alerts through general developers will increase the risk of misdiagnosis and slow repair. As a result, there is an urgent demand of the automated systems that can help developers to obtain quick, secure and standardized diagnosis of vulnerabilities.

The latest developments, mainly encouraged by Large Language Models and their derivatives, have demonstrated substantial potential on particular security-related duties in academic communities. These achievements highlight the technical viability of implementing generative algorithms to analyze code data through strong APIs such as Google Gemini and architecture such as Retrieval Augmented Generation. Nevertheless, multiple challenges exist in the way of such bright experimental scripts into generally accepted and experimentally validated enterprise-level tools and these are limitations of the context window, model hallucinations, workflow integration, and engineering trust.

2.2 Existing Solutions and Their Drawbacks

Although a great deal of research has been undertaken on the topic, detailed and potentially workable intelligence provisions to analyze codebase are still wanton, especially an open domain platform that effectively incorporates a plurality of examining methodologies.

A lot of current scholarly solutions exhibit competence but usually do so in a task-oriented manner. For instance:

- **Static Code Parsers:** Rule based scanners. Traditional scanners have been used, but highly rigid tools are usually unable to provide high level comprehension needed to be useful in an enterprise and runtime verification is not generally provided with these systems.
- **Specific Disease Detection:** Systems have demonstrated specific endpoint live anomaly detection. These tools however are usually ignorant of the underlying code and severely deficient in the localization of the discovery to a particular piece of code, rendering them not actionable in terms of development. The radius of errors that can be identified is often also limited, excluding theoretical logic errors.

The drawback of this fragmentation is one of key limitations, systems are exceptional in either steady parsing or active observation, but seldom combine these into a clinically useful developer overview. Strong hybrid solutions that would integrate structural analysis and active validation of traces do not exist in the open marketplace. Mapping theoretical gaps between code defects and active network edges and execution paths is a major gap that is unmet because this localization is a core pillar of speedy amelioration.

Another important bottleneck is data context and standardization. Although baseline vulnerability databases are a good benchmark, they frequently do not have detailed run traces of the types of frameworks that occur in practice. Interestingly, establishing contextual environments, which combine rigid logic with dynamic telemetry, is an infrastructural undertaking, which impedes any advancement.

SonarQube, Checkmarx and Veracode have featured as commercial providers of cloud based code analysis. Although they are powerful, they have the disadvantages that plague proprietary systems.

- **Price and Availability:** Subscription pricing restricts cost, particularly in low-resource communities.
- **Failure to be Transparent:** The closed boxes are operated by strict sets of rules that are not disclosed and therefore it can only be examined through independent verification, which is problematic because of high amounts of false positive alerts.
- **Limited Customization:** Users do not have the ability to customize these systems or integrate them into the custom runtime environment.

Lastly, much of the specialized security tools suffer from poor usability. Academic prototypes as well as commercial software are commonly characterized by intricate reporting dashboards that demand extensive security training posing an adoption barrier. Moreover, functionalities to support the needs of a variety of developers, i.e., conversational explanation or contextual remediation guidance, are often ignored. Many of the legacy tools are also not explainable, which poses an obstacle to building trust, where developers are not willing to trust in hard and fast alerts with no clear implementation reasons. The present environment is therefore an

unintegrated ecosystem which requires more, holistic, developer friendly, contextual and explainable solution.

2.3 Proposed Project

The recent progress in the field of generative artificial intelligence, specifically, the areas of highly capable language models, based on strong API frameworks, has the potential to address much of the shortcomings of previous endeavors. Through these developments a software auditing diagnostic support system namely MasSecEval is developed taking an integrated usable and informative way and suggesting a common framework to manage both the static and dynamic diagnostic tasks simultaneously.

The MasSecEval has been built on a distributed microservices framework and deployed due to its high reliability in performance and ability to process complex analytical short messages through an asynchronous queue system. This enables simultaneous code ingestion, semantic vectorization and fine-grained trace interception which is essential to the further vulnerability mapping process.

The major novelty is the integrated multi stage pipeline and mapping algorithm of MasSecEval. With the use of deterministic logic auditing through individual worker functions, database embedding and OpenTelemetry traces, the system offers a summary with practical relevance: theoretically identified software vulnerabilities specifically connected to particular execution spans at runtime. This counters the critical mapping defect with the past divisions of a security initiative.

Moreover MasSecEval puts supreme importance on the usability and contemporary software interaction. The Next.js frontend is to be constructed to explicitly target small programming with its conversational chatbot features, supported by Retrieval Augmented Generation, understandable data tables in a visible format, accessible features, and deep explainability modules that contribute to transparency and trust. The reason behind this focus is to make the system work effectively in different development environments by software experts.

MasSecEval leverages OpenTelemetry protocols and adds a smart vector database provided by Qdrant to support the contextual requirements with data such as clinically controls mappings that are not present in the latest statical scanners. The

strategy is broad in that it enables the testing of the advanced generative architectures on application security tasks that are of great relevance.

Overall MasSecEval represents a considerable improvement as it offers the integration of developed generative analysis in a package that allows precise mapping of static data into dynamic data to be presented in a highly usable developer friendly web interface. This solution is to offer a more thorough, efficient, understanding, and available security instrument to the current development, especially useful in the agile engineering setting.

CHAPTER 3 METHODOLOGY

3.1 Product Perspective

MasSecEval can be conceptualized as a diagnostic support system which is an artificial intelligence based product that is specifically tailored to work with software engineers and security professional who are deciphering application vulnerabilities. It does not intend to replicate the security auditor judgment but to supplement their competencies by automating major analytical operations to increase efficiency and may result in improved diagnostic consistency. The time consuming and inherent risk of diagnostic burnout associated with manual code interpretation is dealt with in the system by offering rapid standardized analysis.

The main value proposal is in its integrated nature: handling an uploaded codebase to identify syntax errors, work out execution paths, particular predefined security conditions as per the recommendations of OWASP, and, most importantly, align those static conditions with the dynamically executed traces. MasSecEval insists on usability and uses a developer-friendly web interface that is based on Next.js and complies with the principles of modern Human Computer Interaction. This commitment to usability contrasts with more classical specialized command line software since advanced security diagnostics become more accessible to general developers. It has a flexible deployment system architecture, which is coordinated mostly through Docker containers and can support a wide range of corporate environments. Finally, MasSecEval aims to simplify the continuous integration processes, enable prompt vulnerability detection, and empower the developers with effective, precise, and local diagnostic data.

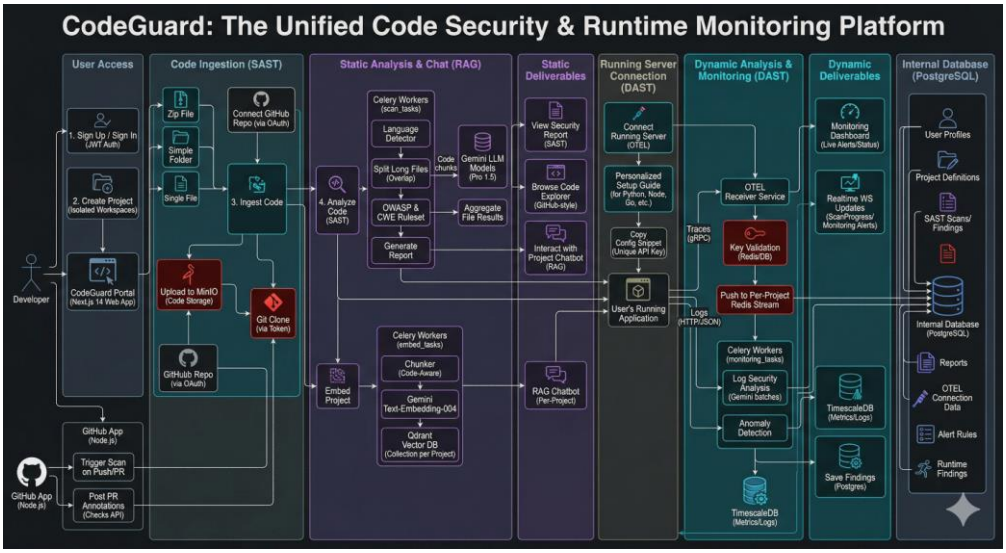


Figure 3.1: Overall Functionality of MasSecEval

3.2 Methods and Approaches

The methodology used in MasSecEval is a mix of cutting-edge AI code analysis algorithms and powerful web technologies and distributed message brokers used to deliver the system and communicate with the users.

The identified data enrichment and embedding:

The input project files are preprocessed using necessary preprocesses before the analysis by the generative models to guarantee uniformity and maximized use of tokens. This consists of:

- **File Extraction and Sanitization:** Parsing uploaded archives or cloning remote repositories, performing a specific procedure of explicitly filtering out irrelevant binaries and dependency folders in order to stabilize the processing speed.
- **Contextual Chunking:** The large source code files are broken down into semantically meaningful overlapping fragments required by the embedding model input layer thereby ensuring accurate vectorization over the dataset.
- **Codebase Storage:** Used MinIO object storage which provides secure and isolated storage of user uploaded repositories and serves as an S3 compatible internal vault of the raw static files.
- **Dynamic Telemetry Ingestion:** A special OpenTelemetry receiver was to capture live execution measurements. This component serves as a listening

post, where it obtains active limits of HTTP requests, database queries, and memory states of the deployed target applications.

Contextual Knowledge Base: This used a vector database constructed specifically to support contextual developer assistance, which indexes the chunked source code in a Qdrant database. This will allow the conversational assistant to remember certain functional context when user queries are asked.

3.2.1 Source Code Preprocessing

Input project files are first subject to critical preprocessing procedures prior to processing by the generative models to guarantee uniformity and efficient utilization of tokens. Here is:

- **File Extraction and Sanitization:** Scanning uploaded archives or cloning remote repositories, taking a deliberate sampling of binaries and dependency folders to avoid garbage as well as to stabilize processing speed.
- **Contextual Chunking:** The encoding model input layer breaks source code files (large file, typically) into semantically significant overlapping chunks that are needed by the input layer, to provide exact vectorization of the entire data set.

The capability to enrich and embed data:

In order to augment the useful analytical potential of the chatbot and enhance the strength of the reasoning with regards to the challenging logic inquiries, data vectorization methodologies were utilized. The system has particular embedding models that convert textual codes into numerical arrays to enable the retrieval augmented generation pipeline to do the mathematical similarity searches on the whole project structure.

3.2.2 Model Architecture

The MasSecEval uses a distributed pipeline of special worker nodes coordinated with Redis to use as a message broker, and run using the Python Celery framework.

- **Architecture:** The analytical backend is built on top of standard REST APIs but introduces an additional, parallel, worker model to execute asynchronous

work and rapid frontend queries, and can be scaled well to long running scans of codebase.

- **Selection of Generative Engine:** Google Gemini model configuration was chosen to be the main reasoning engine. This is an enhanced large language model that enhances semantic representational strength which is important in reflecting various and intricate logic defects in the current web structures.
- **Retrieval Augmented Generation:** This architecture is based on the generative engine and adds contextual cues to the engine, allowing localized anomalies to be effectively detected without hallucinating generic code structures.
- **Worker Configuration:** All analytical tasks were assigned a solo pool configuration in terms of absolute execution stability. The system manages huge repositories by pushing parsing and database migrations, as well as active scanning activities, to different dedicated worker threads. In PostgreSQL, state management was preserved to track the progress and guarantee data durability.

The pipeline used for the inference and mapping consists of the following components:

In a dynamic scan, a codebase uploaded is sequentially scanned by using the deterministic auditing prompts. A custom relational mapping algorithm is then applied to the outputs that consist of file paths, severity of vulnerabilities and remediation snippets. This algorithm determines the structural overlap between the theoretically identified static flaws and the live trace identifiers found in the open telemetry receiver. It correctly correlates each identified static condition with the identifying dynamic network request by cross referencing these datasets in PostgreSQL.

3.2.3 REST API Integration

The interaction between the user interface and the analytical pipeline at the backend is supported by a powerful API. This FastAPI-driven layer receives HTTP requests related to the ingestion of the project, invokes an asynchronous scanning pipeline and delivers the formatted diagnostic findings to the Next.js frontend. A

reverse proxy based on Nginx forwards customer traffic safely among the separate microservices.

3.3 Techniques

3.3.1 Generative Artificial Intelligence

The fundamental protocol of analysis consists of prompting large language models with deterministic instructions that are rigorously strictly speaking. The model also learns to map inputs like raw blocks of source code onto the outputs in highly structured formats like localized vulnerability reports, which is an advanced semantic reasoning scenario.

3.3.2 Large Language Models

The study employs highly parameterized neural networks in the form of transformer architectures that fall under the domain of natural language processing and semantic understanding approaches.

3.3.3 Retrieval Augmented Generation

The architecture is used in contextual assistance where the model does not only find the general security concepts, but also provides the specific answers to the code that are based solely on the user project. It is specifically relevant when it comes to the prevention of generative hallucinations.

3.3.4 Static Application Security Testing

This method is used to read the raw code text and present the candidate area that may contain a security vulnerability and then refinements of classification and severity are done according to the established OWASP parameters.

3.3.5 Dynamic Application Security Testing

The model can both identify memory faults and active database injection paths effectively because it uses dynamic tracing to draw live execution metrics of various deployment situations. It is among the primary aids in proofing theories of security warnings.

3.3.6 Asynchronous Task Queues

Use of message brokers such as Redis and task processors such as Celery suggests a more asynchronous architecture where the system is unburdened of heavy computer processes to background processes so that it keeps the frontend interface highly responsive.

3.3.7 Vector Similarity Search

It aids in matching user natural language queries to the indexed database embeddings with a high degree of accuracy and improves on the context accuracy of chatbot responses is a highly important operation in exploring massive undocumented codebases.

3.3.8 Loss Functions

Instead of loss functions used in training, this system uses deterministic relational mapping:

- **Foreign Key Binding:** Used for associating dynamic traces to static scans.
- **JSON Extraction:** For safely parsing the structured LLM vulnerability output.
- **Telemetry Parsing:** For decoding OpenTelemetry protocol buffers into readable span data.

3.4 Tools and Technologies

The development and deployment of MasSecEval involved a combination of specialized analytical tools and modern web technologies:

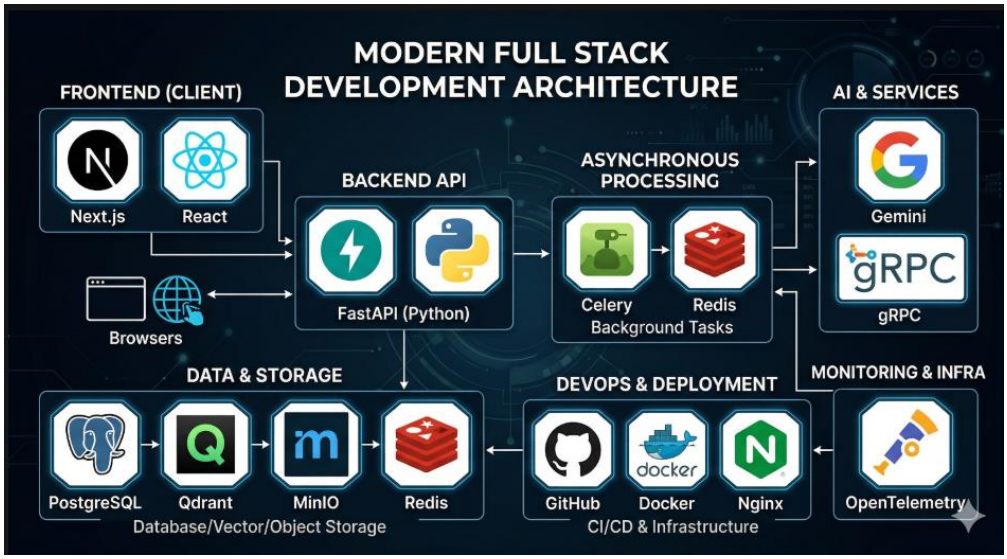


Figure 3.2: Tech Stack of MasSecEval

- **Artificial Intelligence & Telemetry:**
  - Core Generative Engine: Google Gemini API.
  - Telemetry Standard: OpenTelemetry.
  - Vector Embedding: Gemini Embedding Models.
- **Backend Development:**
  - API Framework: FastAPI.
  - Task Processor: Celery.
  - Message Broker: Redis.
  - Programming Language: Python.
- **Frontend Development:**
  - Core Framework: Next.js.
  - Styling: Tailwind CSS.
  - UI Components: React DOM.
  - Programming Languages: JavaScript, TypeScript.
- **Database & Storage:**
  - Relational Database: PostgreSQL.
  - Vector Database: Qdrant.
  - Time Series Database: TimescaleDB.
  - Object Storage: MinIO.
- **Development & Deployment Tools:**
  - Container Orchestration: Docker Compose.
  - Web Server: Nginx.
  - SSL Certification: Certbot.
  - Version Control: Git and GitHub.

CHAPTER 4 SOFTWARE REQUIREMENTS SPECIFICATION

4.1 Overall Description

4.1.1 Product Perspective

The Multi Agent System Security Evaluator project is a new stand-alone product offering innovative and enhanced generative artificial intelligent frameworks and a recognizing and user friendly interface to help software engineering practitioners in diagnosis of vulnerabilities in applications. The software is not designed to substitute the human auditor; it is to supplement the currently existing security testing procedures and as a diagnostic tool, it aims to make complex code analysis much simpler and more precise.

The product may be situated in a more expanded software security testing paradigm, which will capitalize on the prospect of large language models to spur secure deployment events. It is developed on a contemporary distributed architecture which uses Next.js on frontend and a Python FastAPI backend to facilitate smooth communication between the analytical engines and the web application. The main component is the intelligence model, which is implemented with the help of the Google Gemini architecture that is particularly triggered with the purpose to find severe software flaws in accordance with the OWASP rules and regulations.

Major components that the system architecture contains are as follows:

- **User Interface:** The user interface as a developer and security analyst would like to see it is built with Next.js.
- **Backend Server:** Python and FastAPI handle the user requests, routing of data and communication with the worker nodes.
- **Analytical Pipeline:** This is an independent asynchronous worker module that was coded in Celery and Redis and handles the heavy work of code parsing and AI interactions.
- **Database Infrastructure:** A powerful stack using PostgreSQL to store relational data projects, MinIO to store object data in source code, and Qdrant to administer vector embeddings.

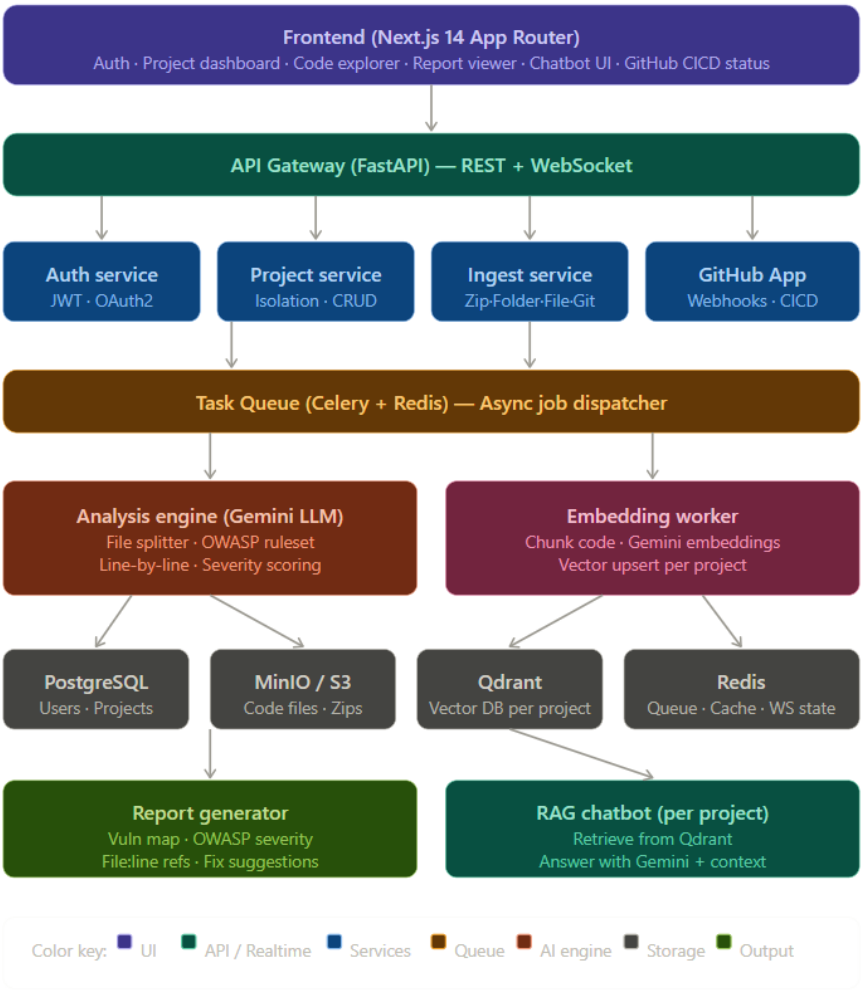


Figure 4.1: System Architecture of MasSecEval

4.1.2 Product Functions

The MasSecEval project will perform the following main functions:

- **User Registration:** The administrators will be able to enter the user information to grant accounts to developers and security personnel.
- **Project Ingestion:** Developers are able to post archive source codes or store remote repositories in the system.
- **AI Security Processing:** The program will scan the uploaded pieces of code with the help of the Gemini model to detect fixed logic states and syntax errors.
- **Telemetry Tracing:** The system will actively monitor OpenTelemetry traces in deployed applications to trace active paths of runtime.

- **Results Dashboard:** Within view of project records, auditors are able to manipulate the overlap of theoretically static flaws with responsive dynamic execution spans.
- **How it works:** The system will intelligently generate particular code refactoring suggestions depending on the intelligence outputs.
- **Contextual Chatbot:** This interface of a Retrieval Augmented Generation will enable developers to chat directly with their codebase to gain better insight.
- **Data Export:** The system will support the sharing of security reports through the standard file format.

4.1.3 User Classes and Characteristics

The application has the following major classes of users:

A) Software Developer:

- Frequency of Use: Daily.
- Functions: Project ingestion, comparing code, vulnerability scanning vs Remediation help, querying the RAG chatbot.
- Technical Experience: Knowledge of computer literacy, knowledge of coding frameworks, and may not have even basic cybersecurity knowledge.
- Responsibilities: Maintaining the scanning of new code commits, fixing flagged code vulnerabilities prior to production.

B) Security Analyst:

- Frequency of Use: Varies, depending on active deployment cycles.
- Functions Utilized: Understanding project dashboards, review of combinative reports of both static and dynamic intelligence, organizational security rule management.
- Technical Skills: Extremely high; profound knowledge of application architecture, OWASP structures, and network monitoring.
- Responsibilities: Auditing conditions, using the data presented, prioritizing on severe vulnerabilities, as well as recording security sign offs.

4.1.4 Operating Environment

- The software will work based on a corporate or cloud based deployed environment and use the following components:
- **Hardware Platform:**
  - The accessibility of desktop and laptop computers by the developers and the analysts that are viewing the web interface.
  - Cloud-based server and high worker processing would be containerized.
- **Operating Systems:**
  - Frontend desktop support on Windows 10 or macOS.
  - Docker environments hosted on cloud based servers with Linux distribution (Ubuntu distribution).
- **Software Components:**
  - Next.js & Tailwind CSS to render a client on the frontend.
  - The central backend routing is done with FastAPI and Python.
  - Celery and Redis for asynchronous task orchestration.
  - PostgreSQL, Qdrant, and MinIO to support data persistence and storage.
  - Nginx as a reverse proxy server that supports secure routing.

4.1.5 Design and Implementation Constraints

The creation of the MasSecEval application will be under a number of constraints:

- **Security Policies:** Strict compliance with internal corporate IP protection rules, ensuring that uploaded source code is isolated and never leaked across tenant boundaries.
- **Processing Limitations:** Compatibility with the rate limits and token windows enforced by the external Google Gemini API.
- **Asynchronous Complexity:** Model training and codebase embedding times could affect the prediction speed, requiring robust background worker queues so the frontend does not freeze.
- **Programming Standards:** Adherence to strict Python and JavaScript best coding practice standards and documentation for long term maintainability.

4.1.6 User Documentation

The user guide of the application will comprise:

- **Administrator Manual:** This is a detailed manual that includes details about the Docker Compose deploying, configuration of environment variables, and the setup of Nginx under SSL.
- **User Manual:** Context sensitive hints at how to create a repository, what the telemetry dashboard represents.
- **Documentation:** Revise and update documentation with each new architecture release.
- **Delivery formats:** This will be standard markdown documents and PDF guides.

4.1.7 Assumptions and Dependencies

The application has to be implemented with the following dependencies and assumptions:

- **Third Party APIs:** The API to core generative reasoning tasks of Google Gemini is continually available.
- **Telemetry Standards:** Hypothesis that the target applications under test are instrumented in a manner that emits standard OpenTelemetry HTTP or gRPC traces.
- **Development Environment:** It assumes the existence of a powerful cloud instance with the ability to run numerous Docker containers at the same time.

4.2 External Interface Requirements

4.2.1 User Interfaces

The application will have an easy to use interface that adheres to modern web conventions and a consistent dark mode friendly look across all screens. Logical design of each interface is created with the intent of providing maximal usability as well as providing self-intuitive navigation through complicated data:

Instructions:

Data separation and separate data modules will be used. There are default components which include a side navigation menu of the main actions such as

Dashboard, Scans and settings and the huge main content area to view the snippets of code and traces. The use of whitespaces will remain uniform to avoid readability of thick blocks of code.

GUI Standards and Product Style Guide:

The user interface will be based on the modern application conventions. It will take a strong color scheme with apparent severity indicators such as red as a critical flaw and yellow as a warning. Branding elements of products will be incorporated to maintain identity.

Normal Buttons and Functions:

Generic buttons like Initiate Scan, Cancel and Export will be some of the preset buttons in the predetermined positions by the top right side to be directly accessible by the user. Images of the most frequent actions will be salient and will be positioned in strategic places within data tables.

Error Messages:

Messages of error will be brief, informative and definitive. They are going to be put in prominently visible toast notification component and will always be in the same format. Repository links are going to be inputted with the actual field showing error feedback.

Software Component that needs to interface with a user:

A) Login Module: To secure the authentication pass on.

B) Organization Dashboard: Visualizing running scans and general vulnerability measures.

C) Project View: The main workspace with a repository of particular files, running OpenTelemetry traces, and detected OWASP vulnerabilities.

D) Chat Interface: A conversation box of Retrieval Augmented Generation developer assistant.

4.2.2 Hardware Interfaces

The software product will be communicating with various network tiers to generate smooth operations.

Supported Device Types:

Desktop Computers: They are compatible with the current operating systems through the use of advanced web browsers. The interface will strongly accommodate wide screens because data tables that will support the analysis of the codes are wide.

Interactions between Data and Control:

The software will communicate with the server via normal web inputs. Vulnerability reports and running trace visualizations will be visualized using the client browser as the output data.

Communication Protocols:

- **HTTP and HTTPS:** In the case of web server to client interactions.
- **gRPC:** To add high speed ingestion of dynamic target server telemetry to the OpenTelemetry receiver.

4.3 Software Interfaces

The software product will communicate with internal microservices and external APIs to provide efficient functionality.

4.3.1 Databases

A) The software will be linked to several database systems of unique data domains:

B) PostgreSQL: It is stored in relational data format like user credentials, project data, scan history, and raw telemetry events.

C) Qdrant: This is a specialized type of vector database that all the numerical embeddings of the chunked source code are stored in to drive the semantic search feature.

D) MinIO: An object storage database that is used as S3 compatible vault to store raw zip files and uncompressed code structures safely and securely.

4.3.2 Web Services and APIs

The application will use the external web services to do logic processing:

- **RESTful APIs:** The Next.js frontend will transmit and receive data in the JSON format to the FastAPI backend.

- **Google Gemini API:** Chunked code strings will be sent to Google servers via secure HTTPS and the structured JSON vulnerability assessment will be obtained in return.

4.3.3 Libraries and Frameworks

The project will rely on several frameworks on core functionality:

- **Next.js:** This offers the react based user facing elements in a high rendering fashion of the server.
- **FastAPI:** Server side routing and parsing of OpenTelemetry and incredibly fast Python.
- **Celery:** Supervises the asynchronous allocation of scanning tasks to worker nodes.
- **Tailwind CSS:** The production of a universal frontend style-building language.

4.3.4 Commercial Components

Google Cloud AI Services: The implementation uses all the intelligence generation and all the vectorization of the Gemini Pro and Gemini Embedding models.

4.3.5 Messages and Data Items

The following data items will be moved in between the internal system network:

- **Input:** Immature raw source code text, live trace spans, consisting of memory states and database queries, and natural language user requests.
- **Outgoing Data:** Fixed vulnerability records, remediation example code, and chatbot replies.

4.3.6 Required Services and Communication

Message Broker: Redis will be used to store tasks in memory and reliably give them to available Celery workers so that no scans are lost in high traffic.

4.4 Communications Interfaces

A) API Integration: The system will offer strong internal entry-points to launch scans, scan worker status, and get concluding telemetry correspondence.

B) WebSocket Updates: To avoid the user needing to refresh the page every five minutes when they are scanning a code, WebSockets will be used to update the frontend dashboard with live data.

C) Network Server Communications: The Nginx proxy will safely redirect frontend traffic at port 443 to the backend API at port 8000 and telemetry traffic at port 4318.

4.4.1 Message Formatting

- **JSON:** For the vast majority of client server communications and internal database mapping.
- **Protocol Buffers:** This type of protocol is used by the OpenTelemetry standard to transmit runtime data as quickly and compressed as possible.

4.4.2 Communication Standards

- **HTTPS:** The web based communication between the client and the server uses Let's Encrypt certificates to secure the communication.
- **WebSockets:** WebSockets are used to provide two-way status updates.

4.4.3 Security and Encryption

- **SSL Encryption:** All the internet communication will be encrypted outside the company in order to prevent unauthorized interception of proprietary source code.
- **JWT Tokens:** JSON Web Tokens will be used to manage their secure session and to authorize particular endpoints of the API.

4.4.4 Data Transfer Rates

- **Normal HTTP Requests:** Client requests must be able to cope with common data structures quickly.
- **Big File Uploads:** Big monolithic repository zip file uploads should be gracefully done in streaming listings so that the Nginx proxy does not run out of memory.

4.4.5 Synchronization Mechanisms

Task Polling: WebSockets may malfunction, causing the Next.js client to gracefully transition to periodic polling functionality every five seconds to discover the Celery task status.

4.4.6 Communication Security and Compliance

The boundaries of data have to be followed very strictly. PostgreSQL should only ever permit access to specifically connected projects that are associated with their particular organization ID using a user token.

4.5 System Features

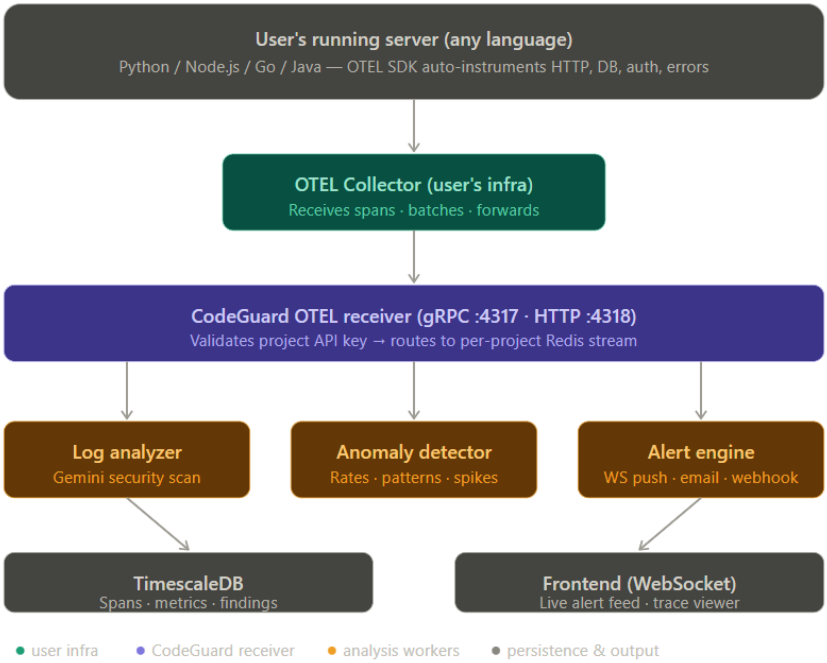


Figure 4.2: User Authentication Flow

4.5.1 Description and Priority

Description: The option enables users to log in confidently in the system with their credentials. Within the system, multi factor authentication will also be supported to guarantee increased security in the organizational deployment.

Priority: High

Benefit: 9

Punishment: 8 (when failing, the developers will not be allowed to view the security dashboards)

Cost: 4

Risk: 7 (security breaches, stealing proprietary code)

4.5.2 Stimulus/Response Sequences

Stimulus: The user is redirected to a log-in page where he/she types in detail such as a password and email address.

Response: System checks the credentials with the database. The user is redirected to their project dashboard on the condition that they are valid. When they happen to be invalid, the error gets displayed and button asked to repeat.

Stimulus: A user opts to use a third-party login option such as Google Workspace or GitHub Enterprise.

Response: This system redirects the user to the third party authentication, retrieves the validation token and redirects the user to the dashboard upon successful execution.

Stimulus: User attempts to log in and opens multi factor authentication.

Response: The system will require a second authentication mechanism including a code delivered to the email or an authenticator application. Access is granted in case of success.

4.5.3 Functional Requirements

- **REQ1:** The system must allow one to log using a valid combination of email and password.
- **REQ2:** The system must provide third party login capabilities through third party services like GitHub and Google.
- **REQ3:** The system must have multi factor authentication meaning that a second method is required on top of primary credentials made by the user.
- **REQ4:** The system will block the account following 5 unsuccessful failed attempts to logout and inform the user of the fact through email containing details of how the account would be unlocked.
- **REQ5:** The system will show the user relevant error messages whenever he/she provides an invalid set of credentials or when he/she is through with the authentication process and the system denies access.

- **REQ6:** The system will record all the attempts to gain access to log in to engage in security auditing.

4.6 Data Entry and Submission of Form

4.6.1 Description and Priority

Description: The feature allows the user to describe the repositories by filling in and submitting electronic forms. It ensures that links to the repositories, branch information, and zip files uploaded are authenticated at the client and server before the generative scanning pipeline is started.

Priority: High

Benefit: 8

Penalty: 7 (submission of penalties with wrong branch data or failure to validate files may cause pipeline failure)

Cost: 3

Risk: 5 (data validation or upload handling errors)

4.6.2 Stimulus/Response Sequences

Stimulus: It is after someone fills out an online form with their link on a GitHub repository and submits it.

Response: The system validates the access to the repository and in the event of a success, stores the metadata into the database and initiates the ingestion. The user is presented with a success message.

Stimulus: User gives a bad link, or unsupported file format.

Response: The invalid fields are highlighted by the system and the user is required to fix the archive format and re-submit.

4.6.3 Functional Requirements

- **REQ1:** This will ensure that the data presented on the form is checked at both the client side and the server side before it is submitted.
- **REQ2:** The system will also display real-time error messages whenever invalid data is entered such as an unreachable repository URL.

- **REQ3:** Data having valid code will be sent to the server by the system and stored in the MinIO object storage in a safe way.
- **REQ4:** The system will give a confirmation message when the project ingestion is successful and the scan begins.
- **REQ5:** When validating data, the system will allow its users to update and re-enter data when the validation fails.
- **REQ6:** The system will keep the records of form submissions and ingestion errors to audit and review them.

4.7 Reporting

4.7.1 Description and Priority

Description: This feature will allow users to generate vulnerability reports. The users will be in a position to filter, sort and display project security information in various formats, distinguishing between theoretical fixed flaws and the verified dynamic telemetry traces.

Priority: Medium

Benefit: 7

Penalty: 5 (reports on inspections or security could not affect deployment decisions in case of poor formatting)

Cost: 6

Risk: 4 (problems in retrieving or displaying complex code mappings)

4.7.2 Stimulus/Response Sequences

Stimulus: A user selects a type of report and refinements (e.g. date range or OWASP severity measurements specifications).

Response: System generates the report by the basis of reduced parameters and presents the vulnerability matrices to the user.

Stimulus: User would like to export the report to PDF or Excel to be used during compliance meetings.

Reply: The system generates the formatted file and initiates the download.

4.7.3 Functional Requirements

- **REQ1:** The users should have access to the functionality to generate custom security reports based on selected filters and criteria.
- **REQ2:** The system must be able to export the reports in different formats such as PDF and CSV.
- **REQ3:** Data such as severity charts should be visualised depending on the report selected in the system.
- **REQ4:** Storing past scans in the system, which can be accessed later on through downloads.
- **REQ5:** The system must be able to handle large sets of vulnerability data without being inefficient in processing these sets within a reasonable period.
- **REQ6:** The system should give error messages in case reports cannot be produced due to illegitimate filters or missing data.

4.8 Other Nonfunctional Requirements

4.8.1 Performance Requirements

The system will be able to meet specific standards in several scenarios to ensure maximum user satisfaction and functionality of engineering teams. The following requirements are used to establish the expected system performance based on the response time, throughput, and resource usage:

A) Response Time:

- **General User Actions:** The system will respond to user actions such as page navigation or repository submission within a normal condition in 2 seconds.
- **Critical Functions:** System tasks such as registering or loading the chatbot should not take more than 1 second to interact with the system without any difficulties.
- **Live Notifications:** WebSockets notification about background Celery worker progress will be delivered within 500 milliseconds of happening.

B) Throughput:

- The system will be able to host 10,000 active concurrent telemetry traces at full load, without the performance of the web servers being affected.

- It will have the capacity to support at least 1,000 transactions/second of the type of form submissions, API calls, and trace intercepts.

C) Data Processing:

- **Massive Reports:** Production and export of huge sized audit reports of all codebase should not take more than 5 seconds.
- The maximum project archive of 100 MB that one can upload at any given network conditions should take not more than 35 seconds.

D) Scalability:

The system will also be scaled horizontally and can maintain additional scan traffic without performance degradation. It needs to support the dynamic provisioning of resources by scaling up Docker containers when development was at its peak.

E) Resource Utilization:

- **CPU Usage:** The system will ensure that the CPU is not used above 80 percent when the system is on maximum load as this will lead to bottlenecks in the containers.
- **Memory Usage:** Memory usage should not exceed 70 percent used memory on all server instances so as to leave sufficiently available headroom to the Qdrant vector database processes.

F) Database Performance:

Single database query performance should occur within 200 milliseconds when loading PostgreSQL in use such as retrieving scan records and doing joins.

G) Network Latency:

The system must support network latency of up to 100ms without any serious performance degradation so that developers on slower network connections can also have a responsive experience.

H) Live System Requirements:

The dashboard must be updated within any 100ms of a network event being intercepted, to accommodate any features that process active OpenTelemetry data.

I) Error Handling:

In the event of API overflow or break down, the error messages should be presented within 1 second and the user should be able to restart his/her scan without delay.

4.8.2 Safety Requirements

The system safety requirements focus on minimizing the threat of loss or damage in addition to compromise of proprietary source code and organizational infrastructure. The following are particular safety issues and preventive measures:

A) Protection of User Data:

The system should comply with any laws concerning the protection of data, in order to securely store any sensitive intellectual property and process them. Transporting and storing the data must be encrypted with industry standard methods of encryption such as AES-256 data at rest and TLS data in transit to ensure that unauthorised users are kept out and code intrusions are prevented.

B) Access Control:

Role based access should be provided where sensitive vulnerability information can be restricted to certain engineering teams. Only privileged administrators should be allowed access to global system components. Multi factor authentication must be enforced on all users in order to enhance security.

C) Error Handling and Fault Tolerance:

The system is supposed to be well-handled in terms of errors as well to prevent background worker crashes which may cause some scans to be lost. Any kind of error must be recorded and directed to the system administrators to be rectified. Producers of Celery should re-queue on their own in case the generative API loses connection.

D) User Interface Safety:

The user interface must be built in such a manner to reduce the possibility of user error. These involve marking buttons distinctly, warning labels on drastic measures such as when deleting an entire project history and enhanced navigation to ensure that one keeps to the point of the action. It should be able to accommodate all users and use accessibility guidelines to do so.

E) Physical Security:

The servers in which the Docker containers will run should be installed in secure enterprise settings with limited access to ensure physical interference. Data centers that are equipped with the system should be in line with pertinent certifications of safety.

F) Environmental Safety:

This system should address the regulations of the environment of data center energy usage, where sustainable practice of cloud computing is observed.

G) Incident Response Plan:

The incident response plan should be prepared and should be maintained in the case of possible security breach or telemetry pipeline failure. This plan must specify the action that will be undertaken during an incident such as communication procedures and escalation processes.

H) Compliance and Certifications:

The system should be subjected to safety checks periodically in order to ascertain that they adhere to rules in the industry and the secure codes. These involve acquisition of the pertinent safety certifications to show compliance with the best practices.

4.8.3 Security Requirements

The security requirements of the system are stressing the confidentiality, integrity, and availability of proprietary code and protection against unauthorized access. Certain security and privacy needs are outlined as following:

A) Identity Authentication When the User is Logged In:

The system should be equipped with strong user authentication mechanism where users will be expected to provide different emails and passwords. All user accounts need to have multi factor authentication. There should be a requirement on password policies including minimum password length, and routine password expiration.

B) Data Encryption:

These sensitive data such as Google Gemini API keys must be encrypted using robust encryption algorithms both in rest and in transit. All communications involving

sensitive information must be encrypted by use of secure programs such as TLS in order to prevent man in the middle attacks.

C) Access Control:

Apply role based access control to ensure that users can only access code repositories which are appropriate to assigned teams. Checks should be undertaken on permissions. AUDIT log entries to track access to sensitive vulnerability reports should be maintained and audit trails to monitor the activity of both the administrator and user.

D) Data Protection and Privacy:

The system is expected to comply with the related legislation in data protection to safeguard the privacy of the organizations and guarantee intellectual property security. The vendor should avoid the use of code snippets to the external language model.

E) Incident Response/Monitoring:

A security incident response plan must be prepared and exercised on a regular basis to outline how to respond to security incidents. Continuous security monitoring tools must be installed so that anomalies in the telemetry stream can be noted.

F) Vulnerability Management:

Security reviews of the MasSecEval codebase itself should be conducted on a regular basis to identify possible vulnerabilities. A process of patch management needs to be implemented to make sure that all Docker images are updated in a timely fashion.

G) Security Training and Awareness:

All those who use the platform should be provided with regular security training programs so that they become aware of security best practices. They should formulate an acceptable use policy that is specific, which outlines the duty of all users.

H) Compliance and Certifications:

The system will be compliant with the relevant security certifications such as the ISO 27001 standards on information security management systems, which is an indication of the commitment to the protection of corporate code.

4.8.4 Software Quality Attributes

The success of the product cherishes the following quality characteristics that add value to both developers and auditors.

A) Usability:

When tested by engineers, the system will be rated as having a usability rating of 85% or more on the System Usability Scale. The user interfaces must provide the users with capability of carrying out common routines such as starting a scan on fewer than 3 clicks to simplify navigation.

B) Reliability:

The system will allow 99.9% uptime in a month, and high availability when ingesting active telemetry. In the case of a failure, recovery time objectives must be less than 30 minutes.

C) Maintainability:

The system codebase should be in Python and Next.js code structure and best practices, the fix and update process should take no more than 24 hours to fix some critical defects.

D) Flexibility:

It will be capable of integrating at least three new external structures within a period of twelve months with little alteration to an architecture that is flexible enough to permit future enhancements.

E) Portability:

The application should be compatible with the existing models of major operating systems and Internet browsers with reduced functionality. The system should be available through the Docker Compose and the deployment time should be the shortest time not exceeding 1 hour in any of the supported Linux.

F) Interoperability:

The product must be able to interact with other systems such as GitHub through standardized RESTful APIs with a response time of less than 200 milliseconds per request. Data exchange formats are supposed to be compatible with standard skills such as JSON.

G) Reusability:

Elements of the codes have to be coded in a format that they can be easily re-used in other modules that cater to other workers and enable them to save time on good code development. The architecture should enable a modular design, allowing taking control of the language model engine and reusing or updating it without interrupting the entire system capabilities.

H) Testability:

The system should have at least 90% code coverage using automated tests so that all important API functionality is testable. The duration to run regression tests should not take more than 1 hour.

I) Robustness:

The system has to be capable of sustaining up to 5,000 simultaneous telemetry tracers without crashes or extreme slowdowns on the receiver port. Error management systems need to provide valuable feedback to the user.

J) Flexibility:

With the system architecture, one should allow the presentation of new prompt engineering rules without major modifications to the current code.

K) Correctness:

After launch, the system will be defect-free with lower than the 1 defect per 1,000 lines of code ratio, high-level functionality, and accuracy of trace mapping.

4.8.5 Business Rules

The following business rules imply the operating rules that shape the functionality of the product. They decide the user permissions and roles:

A) User Roles and Access:

- **Administrators:** They are able to perform all the operations, user, system, and accessing all organizational repositories.
- **Developers:** Have the ability to upload code, start scans, and edit their own team data, but not remote team data.
- **Auditors:** They are capable of reading vulnerability reports and active traces, but are not capable of making structural code changes.

B) Data Modifying and Entry:

During linking of a repository, users must fill all needed fields. Administrative users may only delete whole database of a project.

C) Role Based Access Control:

When creating an account, it will be assigned specific roles that will define the level of access based on the role assigned to users. Any changes of user ranks must be recorded and approved by an administrator.

D) Data Privacy and Confidentiality:

Any proprietary source code is to be kept as a secret. Other team repositories are not expected to view by other users unless they are expressly permitted to.

E) Audit and Compliance:

The user operations, especially those that pertain to project deletion and vulnerability sign offs must be audited by being logged. These records are expected to have user identification, action carried out, and timestamp.

F) Session Management:

The IDs of users should automatically expire after 15 minutes of inactivity where they are required to log out so that the dashboard is safe.

G) Notification Rules:

Necessary actions similar to account modification or long running code scans must be alerted to the user. Serious telemetry pipeline failures should be reported to administrators.

H) Reporting and Data Access:

Only users assigned with specific auditor roles can produce detailed compliance reports with sensitive security data. All designated developers should be able to access basic scan summaries.

I) Password Management:

Strong passwords with standard complexity parameters should be encouraged to the users. The users are supposed to be encouraged to revise their password after every 90 days.

J) Training and Support:

Any new engineer must go through the system documentation before accessing the system, in order to know how to read the dynamic trace mappings.

4.9 Other Requirements

In this section, other product related requirements that had not been discussed previously are documented.

A) Database Requirements:

The program will utilize the PostgreSQL with support to structured queries and will have an intention to expand data tremendously. Backups of the data must be done on a daily basis to avoid losing the important evaluation of security. A special usage of Qdrant will be as a vector store.

B) Internationalization Needs:

The user interface must support multiple languages and it might be extended to other languages in the future to accommodate engineering employees globally.

C) Legal Requirements:

The product shall be able to comply with the intellectual property protection laws. There must be the correct isolation processes on any source code collected and the organization must be aware of how their code is analyzed on the generative model.

D) Reuse Goals:

The project ought to be configured to use pre-existing libraries in OpenTelemetry standard wherever feasible, which assists in fostering efficacy and uniformity along observability pipes.

E) Accessibility Requirements:

The app must be able to comply with the minimal accessibility requirements of WCAG 2.1 Level AA in order to ensure that it can be used by individuals with disabilities. Every user interface element to be provided should be keyboard enabled.

F) Deployment Requirements:

The application should be able to be deployed on cloud environments such as AWS or bare metal local servers on standard Docker Compose setups and must take less than 1 hour to deploy.

Appendix A: Glossary

API: Application Programming Interface; it is a set of rules that allow various software components to interact with one another.

AWS: Amazon Web Services; a cloud computer platform offering a few networking services and computing services.

CI/CD: Continuous Integration and Continuous Deployment; an engineering technique that involves automatically incorporating and testing code modifications.

DAST: Dynamic Application Security Testing; it involves testing a program when it is running to identify active vulnerabilities.

LLM: Large Language Model; an artificial intelligence application that learns how to code due to massive amounts of code data.

OTEL: OpenTelemetry; an open-source observability system that accepts live application traces.

SAST: Static Application Security Testing; performing an analysis of source code without executing it in order to locate possible syntax vulnerabilities.

Appendix B: To Be Determined List

- **TBD1:** Granular repository permission performed on particular repository branches.
- **TBD2:** Select other supported languages to use in the semantic chunking algorithms.
- **TBD3:** Specifications of the third party integration with either Jira or Slack to provide an alert on vulnerabilities.
- **TBD4:** Requirements for compliance with any new generative AI regulations that can come up during the development cycle.
- **TBD5:** Gauges of load testing OpenTelemetry receiver to ensure the system is capable of competing with enterprise load.

CHAPTER 5 DETAILED DESIGN AND ARCHITECTURE

5.1 System Overview

MasSecEval is a powerful multi agent application security environment that will be used to help software developers and security analysts to diagnose and address vulnerabilities in application source code. The system has use of generative artificial intelligence models, and dynamic telemetry algorithms to identify logic errors, structural vulnerability, and operational runtime exploits. The system is developed on a current decoupled stack, composed of Next.js, FastAPI, Python, and PostgreSQL, providing a web based dashboard, and is responsive to allow use on some level of workstation settings. Its application offers real time analysis, feedback and project data management which seeks to improve secure coding practices and better organizational security posture in production setups.

5.1.1 Product Perspective

MasSecEval is a new security diagnostic product that focuses on the software development lifecycle, namely, the area of the DevSecOps. It relies on Large Language Models on Google Gemini, having trained on OWASP Top 10 guidelines, to identify vulnerabilities in the application, such as injection flaws, broken authentication, and security misconfigurations. The modern framework stack facilitates creation of a highly scalable web application so that the solution can be available to development teams around the world. Formulated to fit such continuous integration processes, the application is expected to deliver a user friendly, efficient tool which will help boost the code auditing process and overall applications safety.

5.1.2 Context Diagram

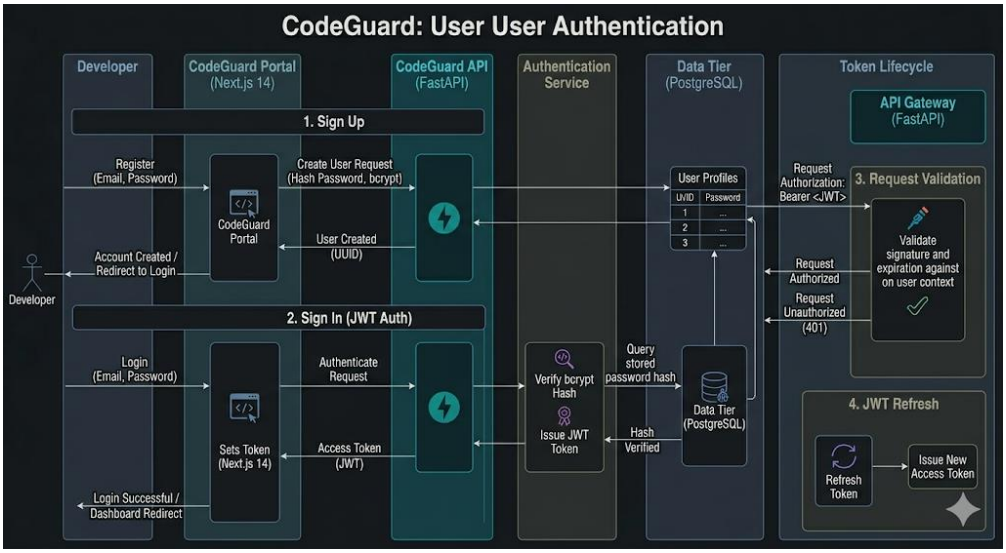


Figure 5.1: MasSecEval Context Diagram, Outlining External Interactions and System Boundaries

5.1.3 Data Flow Diagrams

System Administrator Module

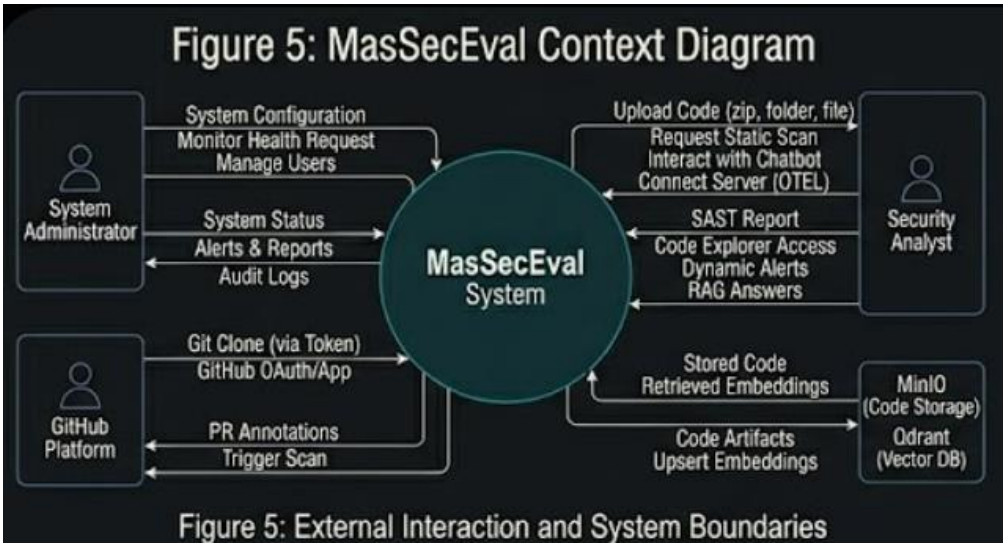


Figure 5.2: Static Data Flow Diagram of MasSecEval

Security Analyst Module

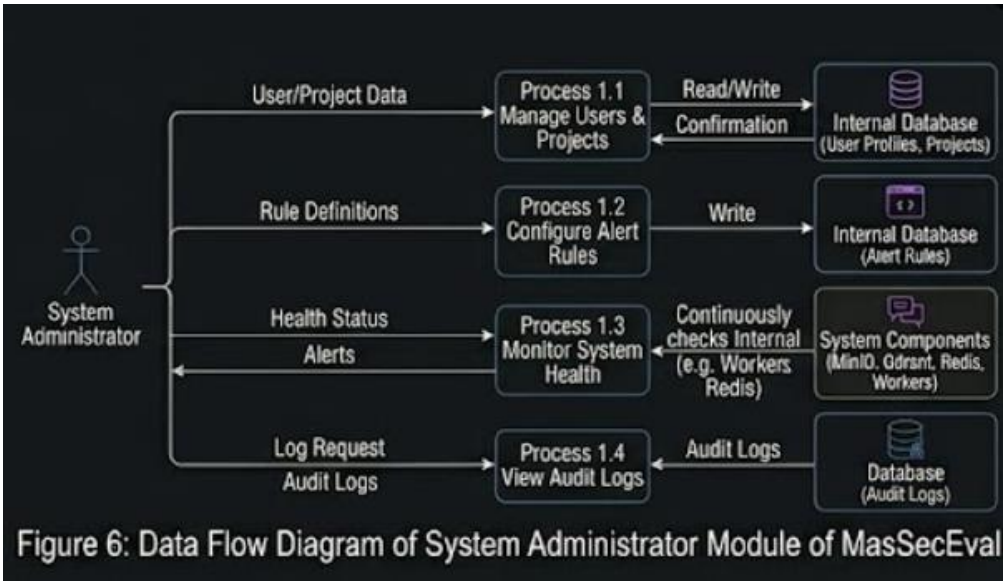


Figure 5.3: Dynamic Data Flow Diagram of MasSecEval

5.1.4 Product Functions

Client Side

- **User Registration:** Developers and security administrators have the ability to register, log into and change their passwords to be able to access the secure system.
- **Source Code Upload:** Users are also able to attach GitHub repositories or compress stores of code to thoroughly analyze it.
- **Vulnerability Detection Feedback:** Having scanned, users get real time predecessor of identified security conditions SQL injections, uncovered credentials or logical routing errors.
- **Progress Tracking:** Tracking of team resolution performance and monitoring of past vulnerability metrics can be done by the security managers.
- **Project Records:** For the project, administrators have access to project repositories, such as telemetry traces, and discovered active threats.
- **View Results:** Developers can access a detailed diagnostic result such as a structural breakdown of the detected code flaws and have confidence in drop in remedies.
- **Conversational Operations:** With assistance developers may be able to search through complex code logic and engage with the application by using a

Retrieval Augmented Generation chat interface to attain context aware assistance.

Server Side

- **User Authentication:** Authenticate and authorize the users so that only authorized individuals can access the proprietary source code and have high levels of tenant confidentiality.
- **PostgreSQL Database:** Store user information, project information, security findings and execution traces in an organized relational format so that it can be easily accessed and managed.
- **Code Processing and Analysis:** Process code chunks with high-level generative AI and abstract syntax tree parsing to find security vulnerabilities.
- **Dynamic Telemetry Receiver:** Use OpenTelemetry listeners to capture the real-time application traffic to compare theoretical established flaws to the dynamic execution trails.

5.2 System Architecture

MasSecEval takes the form of distributed client server architecture that aims at smoothly embedding the artificial intelligence to evaluate security into the development pipelines. The system is supported by a single web platform, built using Next.js and FastAPI as the front and back-end layers respectively and provides the most efficient communication. The user-friendly frontend lets development experts start code scans effortlessly, watch dynamic runtime trace, and handle isolated projects. The powerful Python back-end serves role of authentication, embedding of vectors and running of the Gemini models to detect vulnerabilities. The backend is also connected to PostgreSQL where the relational data is stored and Qdrant where the conversational context is represented in a vector form. The pipeline of analysis that involves Celery asynchronous workers will provide proper and timely evaluation of huge codebases. It is a scalable, dependable, and user-friendly architecture that businesses may use to improve the security of applications.

5.2.1 Architectural Design

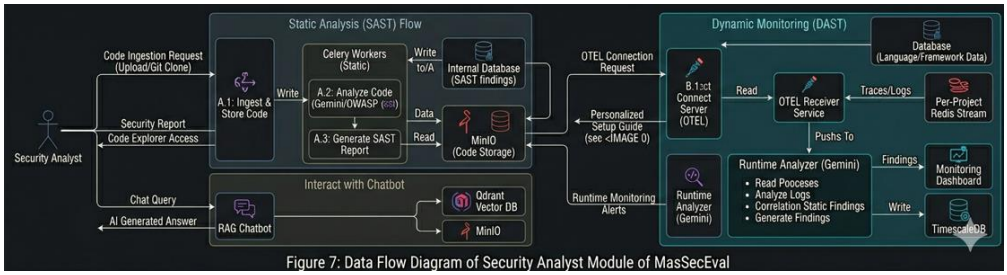


Figure 5.4: Client-Server Architecture Diagram

Client Side

Purpose: This is the user interface where users interface the client side. It enables the communication with the FastAPI backend by the means of the RESTful endpoints.

Components:

- **Signup:** One can create an isolated account.
- **Login:** Authorization of user to gain access into the system.
- **Register Project:** This is a project that enables users such as administrators or team leaders to add new code repositories in the system.
- **Codebase Assessments:** Developers may start the process of deep-based scans of code both in the form of AI scans.
- **Create Security Report:** This creates compliance reports regarding repository health and security posture.
- **Configure Telemetry:** Gives the user the opportunity to attach the live apps to the OpenTelemetry gateway.
- **Talk with RAG Agent:** It is a localized conversational chatbot that is used to get help with codes.
- **Logout:** Finite choice to conclusively end the session.

FastAPI Server

Purpose: Consists of the main backend server, where communication between the client side web application and the asynchronous worker processing units is performed.

Responsibilities:

- **REST API:** FastAPI server is a high performance API that provides a means of communication between the server and the frontend (sending and receiving

data). This API communicates with PostgreSQL and Redis to retrieve projects states.

- **Database Integration:** The server communicates with PostgreSQL which holds information about the project records, users, security findings, and systems metrics through SQLAlchemy ORM.

Celery Worker and OpenTelemetry Server

Purpose: The distributed worker environments handle heavy background work connected to code parsing, vector embedding and processing real time traces.

Responsibilities:

- **Code Preprocessing:** As soon as a repository has been uploaded to the MinIO storage, this preprocesses it to render it fit to be evaluated using Gemini models by chunking and cleaning syntax.
- **Telemetry Ingestion:** HTTP and gRPC messages sent by the applications are intercepted by the dedicated OpenTelemetry receiver, which monitors the paths that were actively used and matches them to static results.

5.2.2 Decomposition Description

Module Decomposition

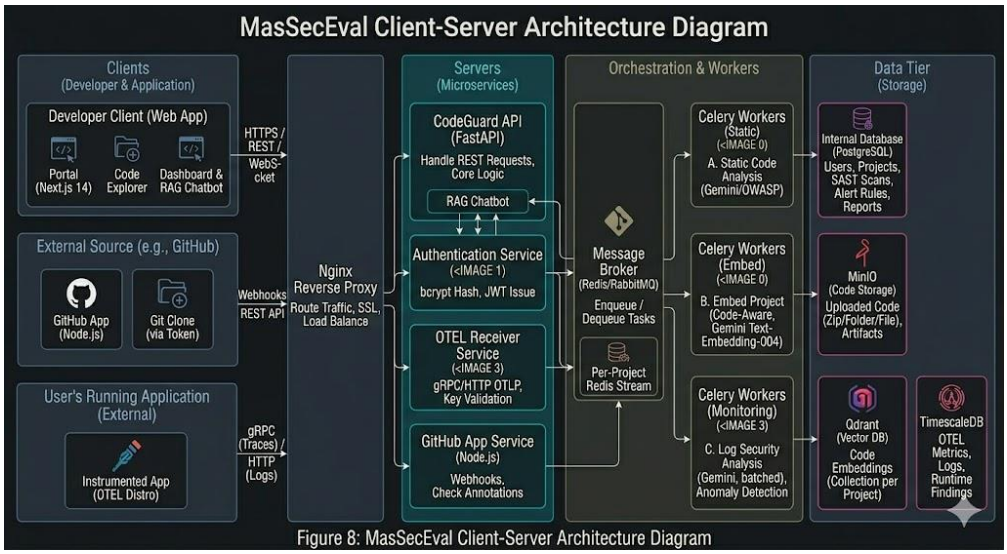


Figure 5.5: Modular Diagram of MasSecEval

Overview of Modules

A. Use Case Diagram

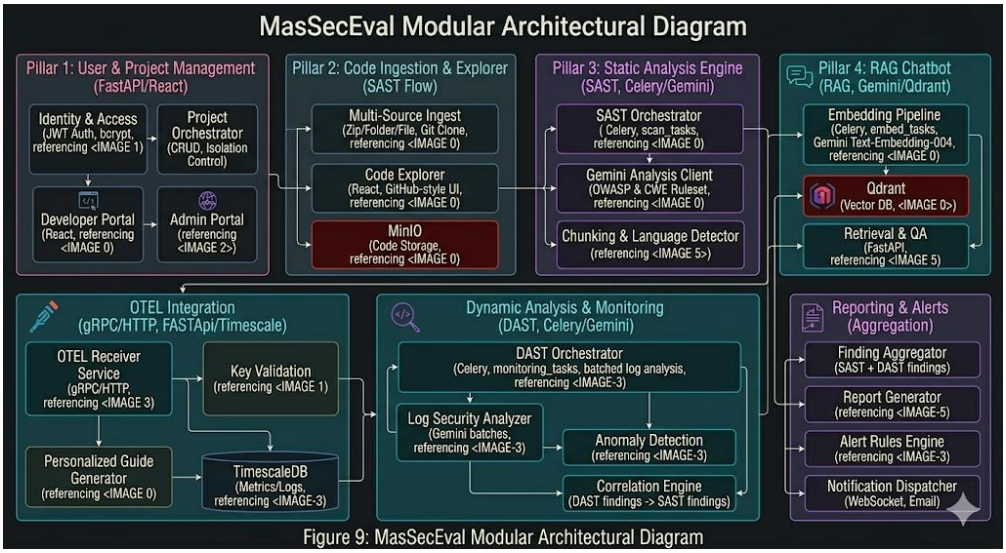


Figure 5.6: Use Case Diagram of MasSecEval

Table 5.1: Use Case 1 — Upload Code Repository

Field	**Description**
Use Case ID	1
Use Case Name	Upload Code Repository
Actors	Developer
Created By	Usman Baig
Last Updated By	Usman Baig
Date Created	14-12-2024
Date Last Updated	14-12-2024
Description	Use case that allows a developer to upload the source code or link a GitHub repository.
Preconditions	Developer should be registered in the database and be logged in with valid credentials.
Post conditions	Developer will move to the scanning dashboard after successfully linking the repository.
Normal Flow	The developer uploads the project archive through the system. The uploaded file is saved in the MinIO object storage. The

	system generates an isolation key to securely separate tenant data. The developer proceeds to initiate the security scan.
Alternative Flows	User can provide a direct URL to automatically clone the repository.
Exception Flows	Invalid account, network timeout, and unsupported repository formats.

Table 5.2: Use Case 2 — Enter Project Details

Field	**Description**
Use Case ID	2
Use Case Name	Enter Project Details
Actors	Developer
Created By	Usman Baig
Last Updated By	Usman Baig
Date Created	14-12-2024
Date Last Updated	14-12-2024
Description	Use case that allows the user to enter technical parameters and framework details of the project.
Preconditions	User should be registered in the database and logged in with valid login credentials.
Post conditions	Project metadata will be saved in PostgreSQL, and the user can proceed to the analysis configuration.
Normal Flow	The developer enters project details including framework version, primary language, and environment variables into the form and submits it. The data is securely saved in the database.
Alternative Flows	The system can auto detect the primary language based on file extensions.

Exception Flows	Database connection error, missing required environment fields.

Table 5.3: Use Case 3 — Assign Security Analyst

Field	Description
Use Case ID	3
Use Case Name	Assign Security Analyst
Actors	Administrator, Security Analyst
Created By	Usman Baig
Last Updated By	Usman Baig
Date Created	14-12-2024
Date Last Updated	14-12-2024
Description	Use case that allows the administrator to assign a specific security analyst to review a project.
Preconditions	Administrator must be logged in with valid credentials, and the project details must already be established.
Post conditions	An analyst will be assigned to the project, and the system will grant them restricted access to the data.
Normal Flow	The administrator selects an analyst from the access control list, assigns them to the repository, and the system records the permission mapping in the database.
Alternative Flows	The administrator can assign entire teams instead of individuals.
Exception Flows	Analyst account inactive, permission mapping database failure.

Table 5.4: Use Case 4 — Analyze Source Code

Field	Description
Use Case ID	4

Use Case Name	Analyze Source Code
Actors	Security Analyst
Created By	Usman Baig
Last Updated By	Usman Baig
Date Created	14-12-2024
Date Last Updated	14-12-2024
Description	Use case that allows the analyst to trigger the generative AI to audit the codebase for vulnerabilities.
Preconditions	The analyst must have access rights, and the source code must be successfully stored in the object storage.
Post conditions	The system will generate structured findings detailing exact code snippets and confident remediations saved in the database.
Normal Flow	The analyst opens the project record, clicks initiate scan, and the Celery workers extract the code. The Gemini model analyzes the chunks and maps vulnerabilities based on OWASP guidelines.
Alternative Flows	The analyst can choose a quick scan targeting only specific directories.
Exception Flows	API key exhaustion, worker node failure, corrupted code archives.

Table 5.5: Use Case 5 — Recheck Dynamic Telemetry

Field	**Description**
Use Case ID	5
Use Case Name	Recheck Dynamic Telemetry
Actors	Security Analyst
Created By	Usman Baig
Last Updated By	Usman Baig
Date Created	14-12-2024

Date Last Updated	14-12-2024
Description	Use case allowing the analyst to verify if theoretically analyzed static flaws map to actively exploited dynamic pathways.
Preconditions	The project must have the OpenTelemetry agent installed and forwarding active runtime traffic to the platform.
Post conditions	The analyst confirms the threat confidence level, adjusting the severity based on live traffic evidence.
Normal Flow	The analyst views a detected SQL injection flaw. They open the telemetry tab and observe live database queries validating if sanitized runtime inputs are bypassing the vulnerable function. The analyst adjusts the finding.
Alternative Flows	The system automatically downgrades the severity if dynamic traces prove the code path is unreachable.
Exception Flows	Telemetry port blocked, Redis stream interruption.

Table 5.6: Use Case 6 — View Infrastructure Health

Field	**Description**
Use Case ID	6
Use Case Name	View Infrastructure Health
Actors	Administrator
Created By	Usman Baig
Last Updated By	Usman Baig
Date Created	14-12-2024
Date Last Updated	14-12-2024
Description	Use case to allow the admin to monitor the status of Docker containers and Nginx routing.
Preconditions	Admin must have root dashboard access.

Post conditions	Admin successfully reviews worker node capacity and API response times.
Normal Flow	The admin navigates to the infrastructure tab and views real time ping statistics for PostgreSQL, Redis, and Qdrant services.
Alternative Flows	Admin can download raw system logs for manual review.
Exception Flows	Monitoring service offline, connection timeout.

Table 5.7: Use Case 7 — Query RAG Chatbot

Field	**Description**
Use Case ID	7
Use Case Name	Query RAG Chatbot
Actors	Developer
Created By	Usman Baig
Last Updated By	Usman Baig
Date Created	14-12-2024
Date Last Updated	14-12-2024
Description	This use case allows developers to ask conversational questions about their codebase and received vulnerabilities.
Preconditions	The code must be vectorized into the Qdrant database under the correct tenant isolation key.
Post conditions	The developer receives accurate, contextually grounded answers regarding specific files or security fixes.
Normal Flow	The developer opens the chat panel. They type a question regarding a specific logic module. The system retrieves vector matches from Qdrant, supplies them to Gemini, and returns a tailored explanation to the developer.
Alternative Flows	The developer asks for a drop in code replacement snippet.

Exception Flows	Vector database unreachable, insufficient context gathered.

Table 5.8: Use Case 8 — Generate Security Report

Field	Description
Use Case ID	8
Use Case Name	Generate Security Report
Actors	Security Analyst
Created By	Usman Baig
Last Updated By	Usman Baig
Date Created	14-12-2024
Date Last Updated	14-12-2024
Description	This use case allows the analyst to export a comprehensive security compliance report.
Preconditions	The static and dynamic scans must be marked as complete in the database.
Post conditions	A consolidated report including OWASP summary charts will be downloaded.
Normal Flow	The analyst reviews all verified findings. They click the generate report button. The system queries PostgreSQL for total critical, high, and medium counts. The PDF is generated and served to the user.
Alternative Flows	The analyst customizes the report to exclude low severity warnings.
Exception Flows	PDF generation library failure, incomplete scan data.

Table 5.9: Use Case 9 — Submit Remediation Approval

Field	Description

Use Case ID	9
Use Case Name	Submit Remediation Approval
Actors	Developer
Created By	Usman Baig
Last Updated By	Usman Baig
Date Created	14-12-2024
Date Last Updated	14-12-2024
Description	Allows the developer to mark a specific finding as resolved after applying the suggested code changes.
Preconditions	Developer is logged in and viewing an active security alert.
Post conditions	The finding status is updated in the database to resolved.
Normal Flow	The developer reviews the suggested code fix. They apply it to their local repository. They click the resolve button on the dashboard, leaving a comment. The system updates the finding entity.
Alternative Flows	The developer marks the finding as a false positive with justification.
Exception Flows	Network disconnect during submission.

B. Sequence Diagrams

I. Developer Module

Figure 5.7: Sequence Diagram for Developer Module

This sequence diagram illustrates the interaction between various components of the MasSecEval architecture: Frontend, FastAPI Backend, and MinIO Object Storage. It captures the flow of actions for adding project details, uploading a codebase archive, generating vector embeddings, and storing relevant data.

A) Actors and Components:

- **Frontend:** User facing interface built on Next.js where the actions originate.
- **Backend:** The Python server side logic that processes requests and communicates with databases.
- **MinIO Storage:** Storage layer for raw source code archives.

B) Process Explanation:

Add Project Details: The process begins when the user adds project metadata in the Frontend. The Frontend sends this data to the Backend via REST APIs.

Send Source Code: The Frontend sends the compressed repository to the Backend. The Backend forwards the payload to MinIO Storage, where it is stored securely under a unique isolation key.

Store Vectors: After storing the code, the Backend triggers a Celery task to chunk the code and send vectors to Qdrant.

Success Notification: Once all operations are successfully completed, the Backend sends a success notification via WebSockets back to the Frontend.

II. Security Analyst Module

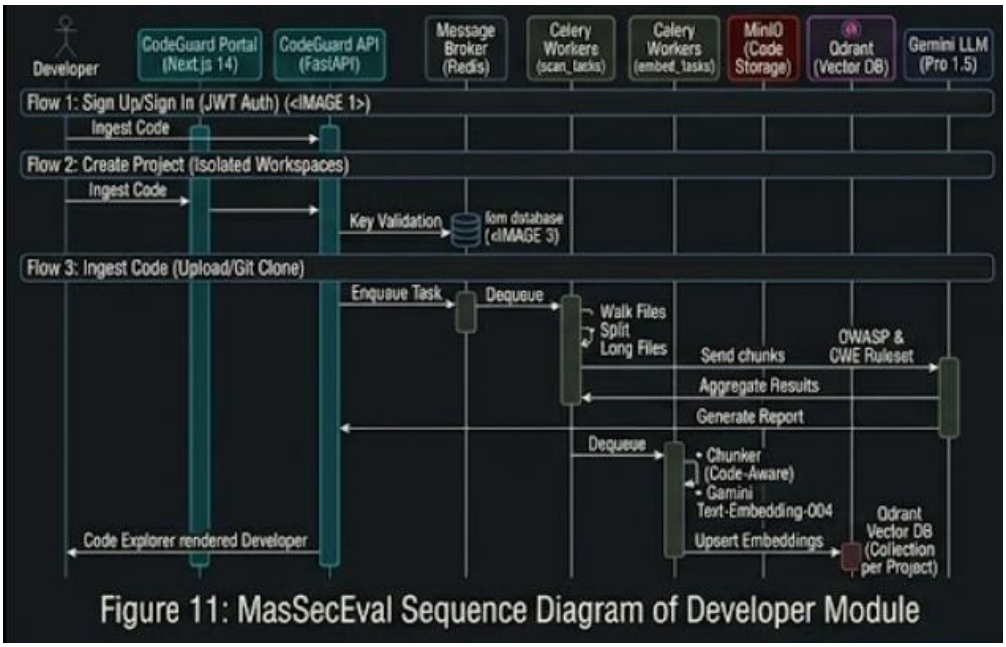


Figure 5.8: Sequence Diagram for Security Analyst Module

The provided sequence diagram depicts a system workflow for a security platform involving an analyst, frontend, backend, LLM model, and PostgreSQL storage.

Analyst Login: The analyst enters their JWT credentials on the frontend. The backend verifies the signature and grants access.

Project Selection: The verified analyst views a specific project from the dashboard. The backend retrieves the repository state.

Code Processing: The retrieved code chunks are sent to the Gemini AI model for explicit deterministic auditing. The model processes the chunks based on OWASP Top 10 guidelines and returns structured JSON findings. The backend stores the findings in the PostgreSQL database.

Diagnosis and Report: The backend sends the findings to the frontend. The analyst reviews the exact offending lines and confident remediations. The analyst approves the report which is then saved for compliance records.

Key Observations:

- The system relies on a high performance FastAPI backend to orchestrate interactions.
- The generative AI plays a crucial role in reducing analytical interpretation time.

C. Activity Diagrams

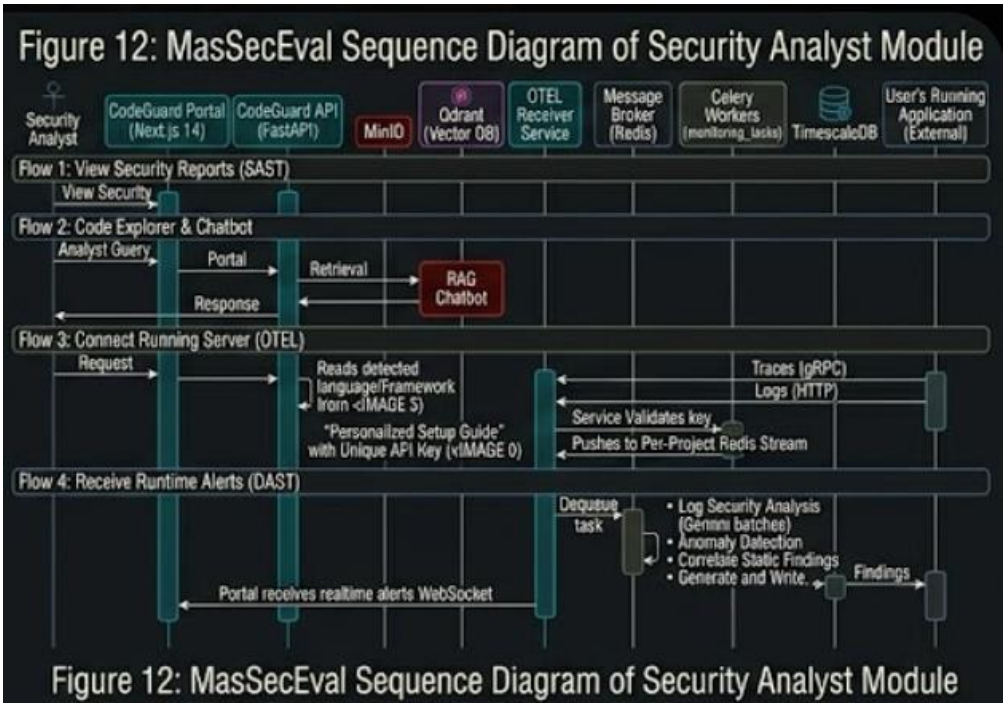


Figure 5.9: Activity Diagram of MasSecEval for Static Testing (Developer)

This activity diagram represents the process flow for a developer managing a repository scan.

- **Start:** The process begins when the developer navigates to the portal.
- **Create Account:** If unregistered, they sign up securely.
- **Sign In:** They authenticate into their tenant space.
- **Upload Code:** The developer uploads the repository.
- **Configure Scan:** They set parameters for SAST and DAST integration.

- **Await Results:** The system orchestrates multi stage workflows via Redis streams.

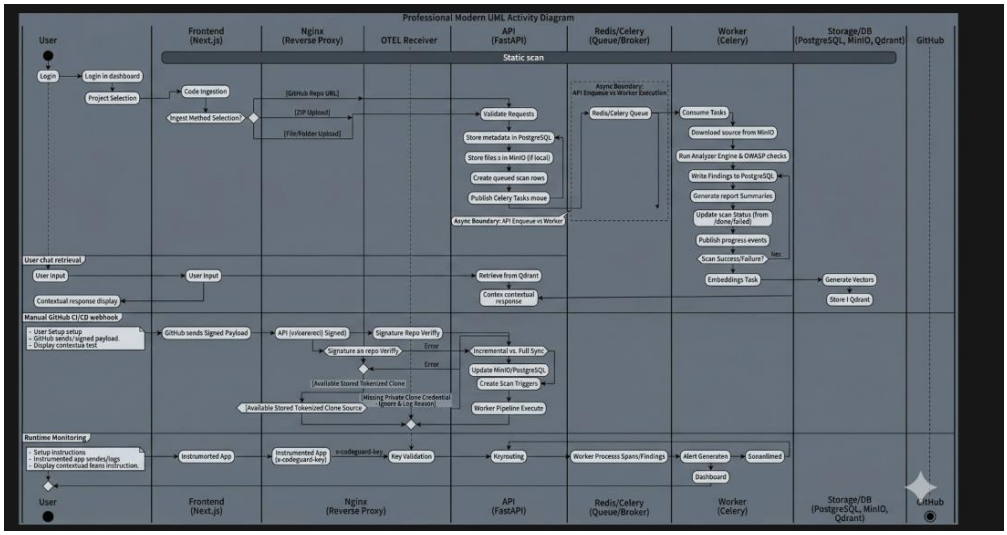


Figure 5.10: Activity Diagram of MasSecEval for Dynamic Testing (Security Analyst)

The flowchart is the flow of the process cycle of analyst tracing validation.

- **Start:** Getting into the system.
- **Sign In:** Strong authentication.
- **Review Static Output:** Analyst verifies theoretical checking of the LLM results.
- **Cross Reference Telemetry:** Analyst monitors the OpenTelemetry stream to confirm whether the flaw can be accessed in running environments or not.
- **Level of Adjustment:** Analyst completes the measure of threat.
- **Save Report:** Analyst completes the audit cycle.

D. Package Diagram

The MasSecEval package diagram shows system modular architecture. The Client package includes the Next.js UI, which is linked to the FastAPI Gateway to handle user requests. The Backend package is the system base, a service that receives requests and ensures the safe handling of data. Database package has PostgreSQL as it provides organized relational tables such as users, projects and safety discoveries. The Worker package is dedicated to Celery-powered asynchronous operations, including text extraction operations, vector embedding using Qdrant, and real time telemetry

processing using the OpenTelemetry receiver. This system allows it to scale and provide MasSecEval with the correct and sound logic flow analysis.

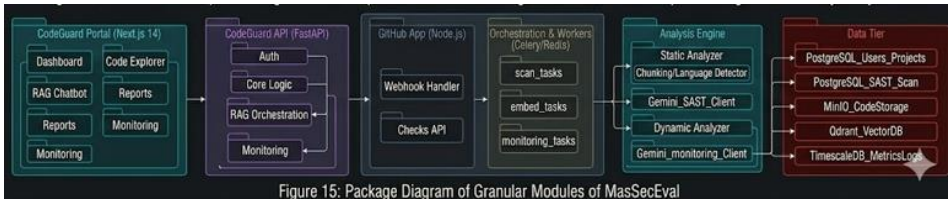


Figure 5.11: Package Diagram Representing Granular Modules of MasSecEval

E. Deployment Diagram

The implementation design is skewed towards the industry-standard containerised setup. Nginx is used in the system as a very efficient reverse proxy to present the Next.js frontend on the Internet, but to route API traffic in a secure manner. Depending on one domain and using Certbot, a single port (port 443) serves all the incoming requests with the help of SSL.

Traffic Flow Rules:

- The root path requests are forwarded to the Next.js frontend that serves on the port of 3000.
- The API has requests that are rewritten and sent to the FastAPI to port 8000.
- Requests that correspond to the telemetry route are sent to the internal OpenTelemetry collector that runs on port 4318. Such a structure will help to isolate all the existing worker services, Redis brokers, and database instances in the internal Docker network and never expose it directly to the external traffic.

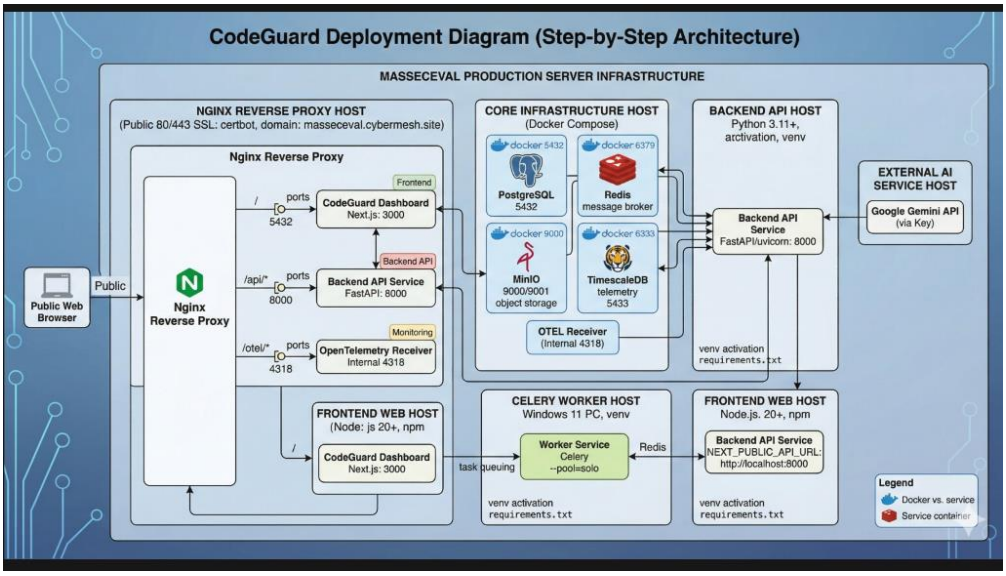


Figure 5.12: Deployment Diagram Representing Flow of MasSecEval

Design Rationale

The architectural rationale behind the MasSecEval design makes use of some important architectural patterns that are aimed at making the platform extremely concurrent, maintainable, and strictly isolated to respond to the complex requirements of enterprise security testing.

Microservices Pattern: The architecture has divided frontend, backend REST API, and background worker processing into individual Docker containers. This guarantees that e.g. heavy processing in LLM or high velocity ingesting telemetry will not tie up the main web interface so that it cannot handle requests made by the user.

Publisher Subscriber Pattern: MasSecEval runs as an evented based system and relies on Redis as a message broker. Once an application trace generates a runtime event, OpenTelemetry receiver emits the payload into the Redis streams. The Celery employees note such a queue and process the anomalies asynchronously to update PostgreSQL and send WebSocket notifications, which provide real time dynamic feedback.

Tenant Isolation Pattern: The security tools need total information privacy. Each scanned resource has its own isolation key. This Universal Unique Identifier creates certain paths to the directories in MinIO and rigid separations in Qdrant and forms physical barriers to deter cross-tenant information leakage.

5.2.3 Data Design

Data Description

The information in MasSecEval is based on relational entity mapping and high dimensional vector representations. These user data include authentication information, project associations, and system measure. The product of the SAST pipeline is the source code artifacts as the input, the backend is based on Gemini models and provides structured logic findings, annotated by line numbers and Common Weakness Enumeration IDs. In parallel, runtime telemetry traces constitute dynamic data, which has been used to cross reference the structure across alerts that are static and spans of active HTTP transactions. All the insights are condensed in strong schematized tables.

Database Schema:

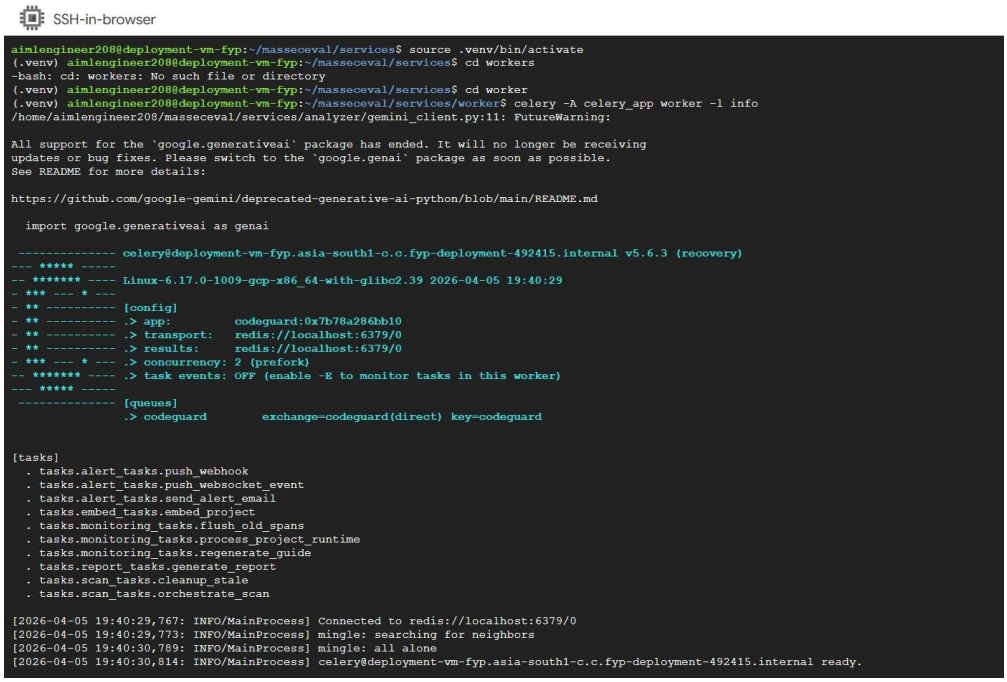


Figure 5.13: Entity Relation Diagram for the Database Entities in MasSecEval

Dataset:

The major dataset that is followed in the analytical intelligence of the MasSecEval is based on the OWASP Top 10 guidelines and extensive Common Weakness Enumeration repositories. Such standardized security dictionaries contain the basic logic rules that the LLM uses to detect misconfigurations. In addition, the system constructs prioritized vectorization datasets in every project with locality. In case code is consumed, the code is converted through embedding models to high dimensional data points. This dynamic project specific data drives the smart developer

chatbot which supports conversational querying inclusive of deep variable, functional and architectural context.

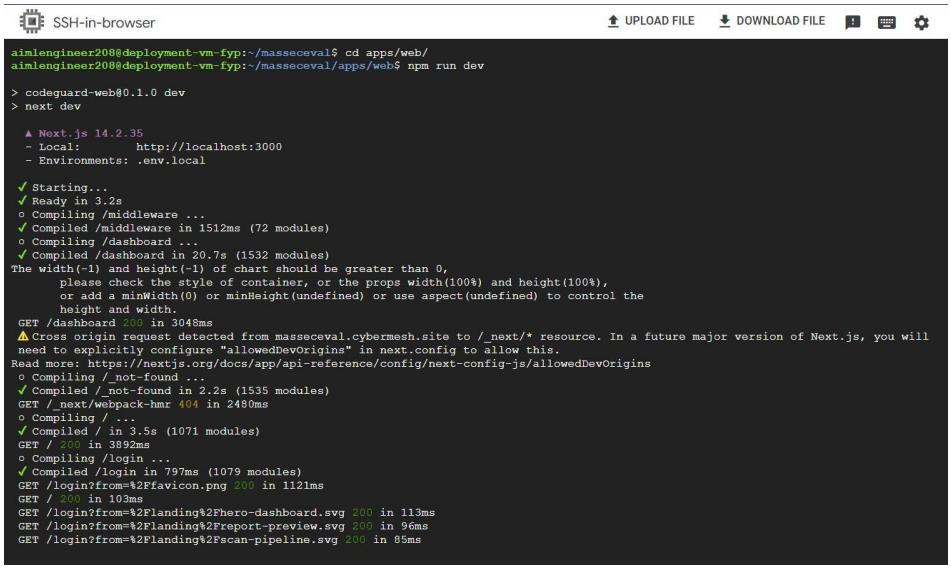


Figure 5.14: A Display of Sample Vulnerability Code Snippets Used For Context

5.2.4 Component Design

The MasSecEval component diagram represents the essential elements and how they relate both in the infrastructure. The Interface is the Next.js Interface. It interacts with the Nginx Gateway in order to safely route requests. The FastAPI router takes care of the flow of logic and the heavy scanning requests are sent to the Celery Task Broker using Redis. Data Handling component takes care of storage, where raw code is pushed to MinIO and structured data to PostgreSQL. The Generative AI Node where system core analytical processing is performed runs deterministic audits. At the same time, the OpenTelemetry Gateway component receives traffic on active traffic, establishing a feedback loop. After analysis, the findings are put in Reporting Repository. The flow of components is connected, and the parts collaborate, to assure seamless user experience and promote DevSecOps integration.

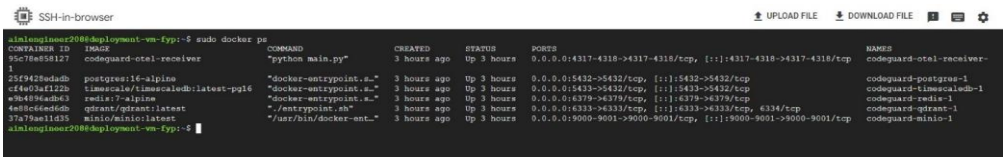


Figure 5.15: Overview of Component Design for MasSecEval

5.2.5 Human Design Interface

Screen Images

A. Login Page

A clean, vulnerable free authentication portal amongst developers and administrators. The portal allows verification of encrypted passwords and future delivery of JSON Web Tokens in case of success in order to redirect users to their separated dashboards.

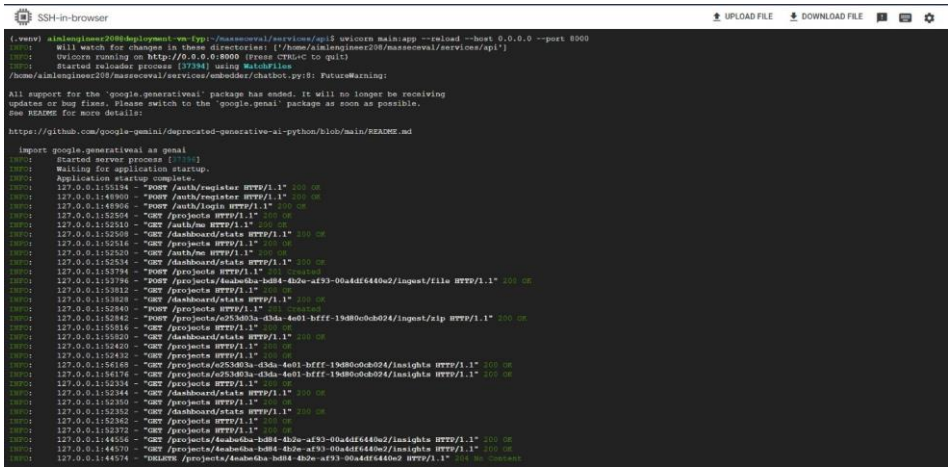


Figure 5.16: Login Page

B. Signup Page

The registration page allows users to establish new accounts. Each registration generates secure database entries and prepares tenant namespaces for future project isolation.

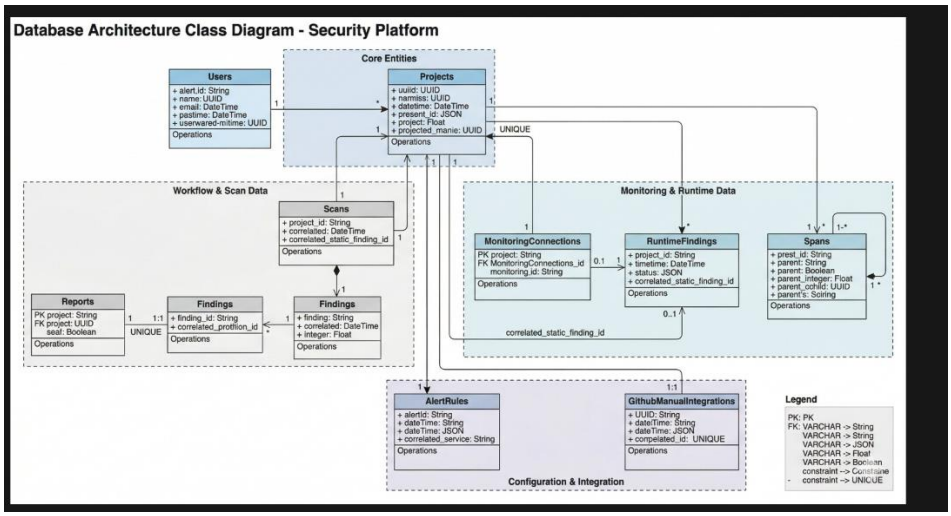


Figure 5.17: Sign Up Page

C. Home Page

A general overview landing page providing insights into the platform capabilities. It highlights the hybrid SAST and DAST architecture without requiring active authentication.

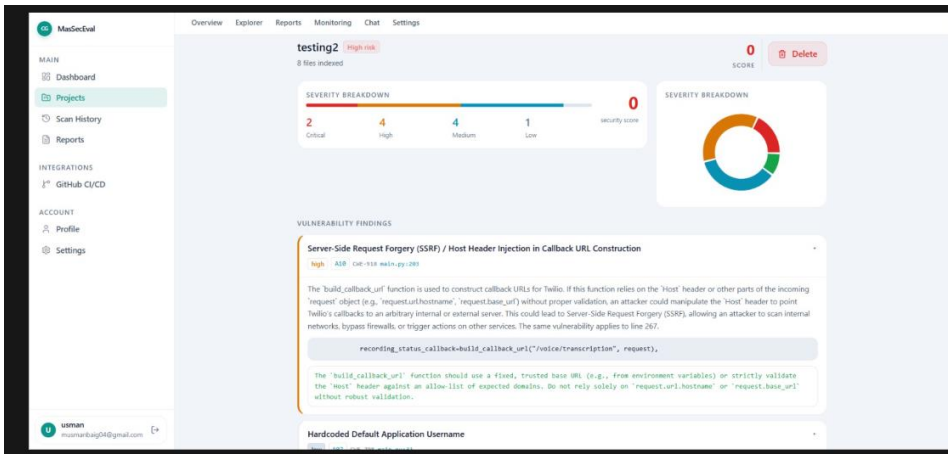


Figure 5.18: Home Page

Developer Module

A. Dashboard

At the main dashboard, developers can see all linked repositories and recent scan metrics. The projects are sorted by activity, displaying quick statistics on critical, high, and medium severity findings discovered during recent evaluations.

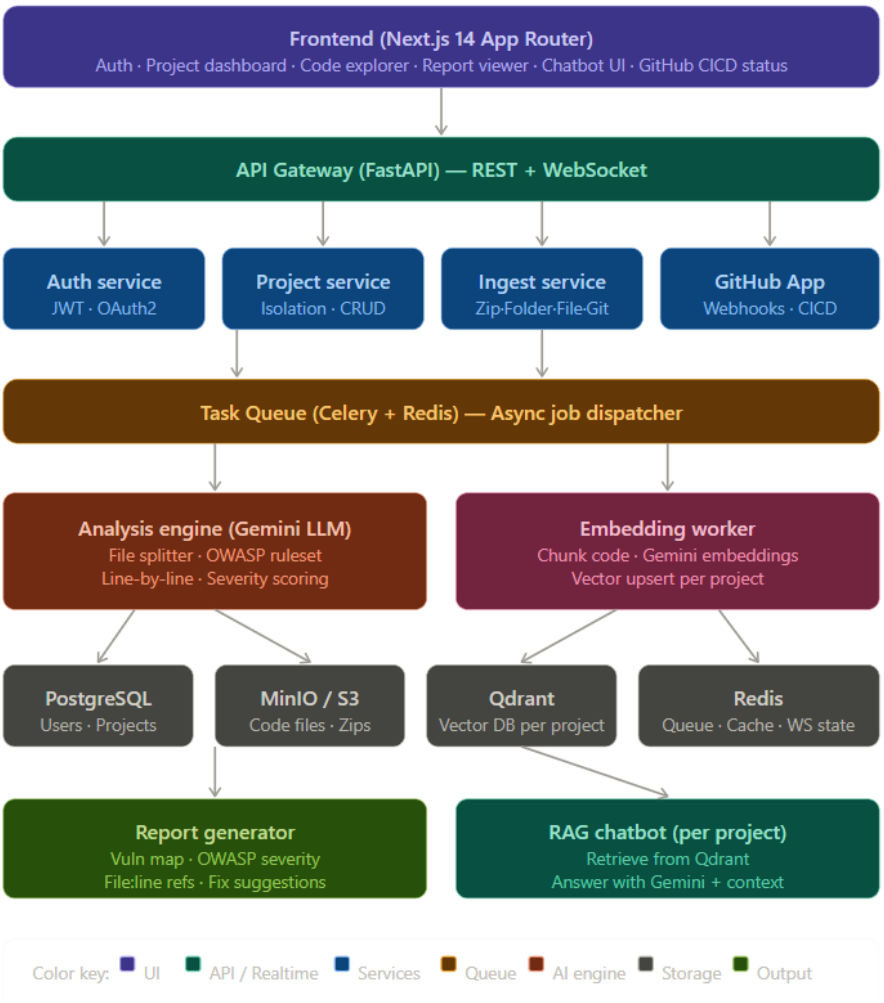


Figure 5.19: Project List Shown to Developer

B. Diagnosis Page

A detailed look at security is presented in the diagnosis page. At the top, a summary of the vulnerability density is given. The interface shows the specific offending fragments of code together with dynamically generated drop in fix suggestions. The developers will be able to examine the Common Weakness Enumeration IDs and determine whether the open telemetry detected the occurrence in a dynamic manner.

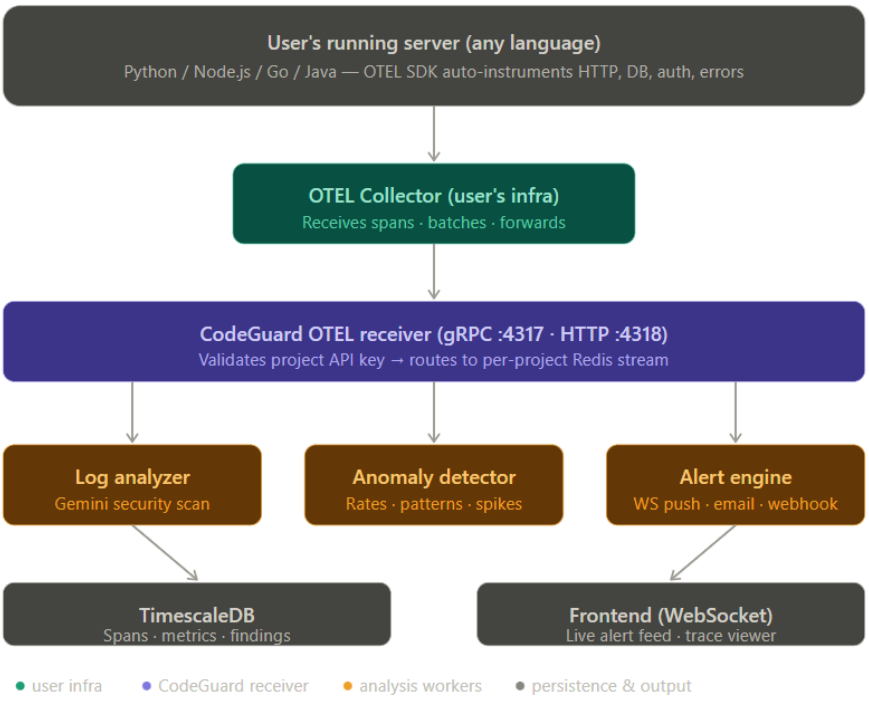


Figure 5.20: Diagnosis Page with Vulnerability Details and Code Snippets

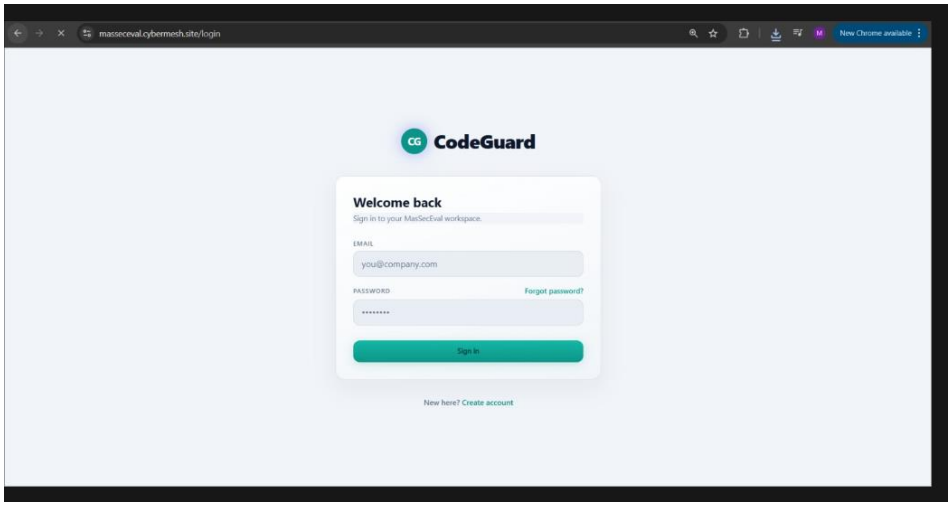


Figure 5.21: Feature for Manual Review and Applying Code Remediations

C. Help Page

Users can interact with the embedded RAG chat interface to ask specific questions about the codebase architecture or seek clarification on complex security vulnerabilities, achieving rapid contextual support.

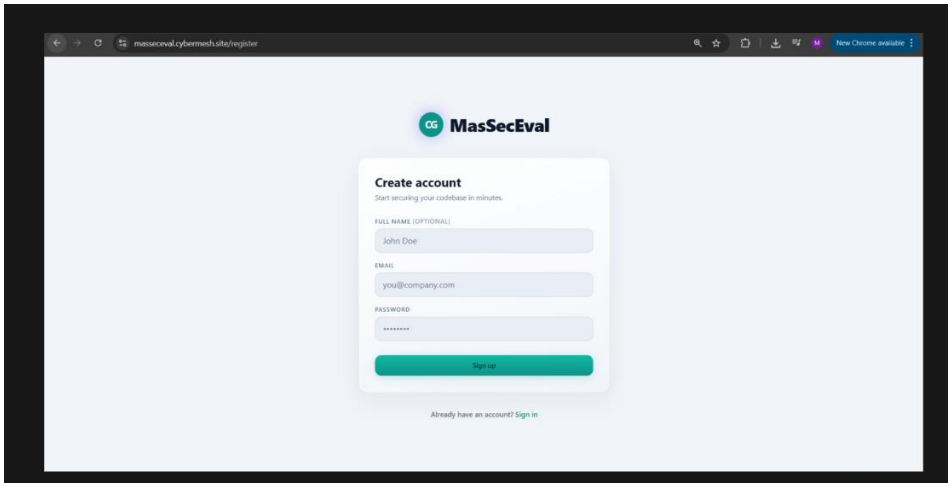


Figure 5.22: Chat Interface Help Page

Administrator Module

A. Register Project Page

The project configuration page where administrators link GitHub repositories, establish OpenTelemetry connection headers, and define the environment boundaries before initiating the hybrid security analysis pipeline.

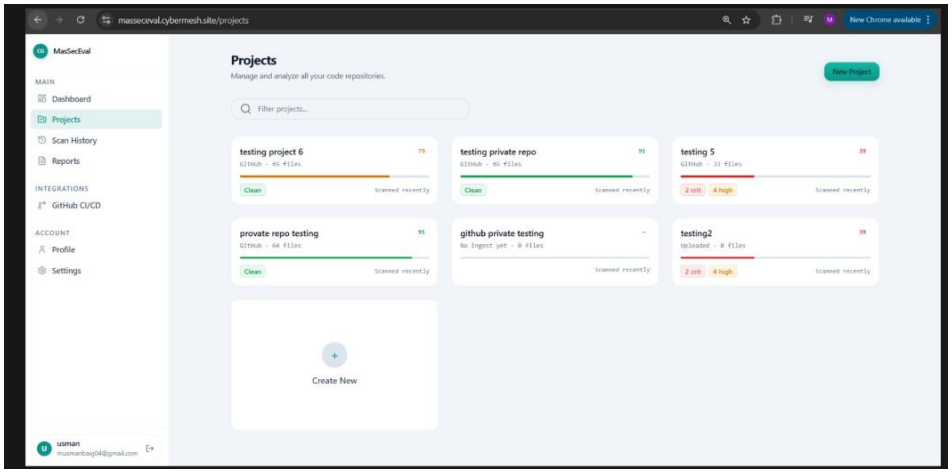


Figure 5.23: Page for Administrator to Enter Project Parameters

CHAPTER 6 CONCLUSION AND FUTURE WORK

6.1 Overview of MasSecEval

MasSecEval is a notable use of artificial intelligence in the software development industry; it is a simple to use, highly effective, and developer focused tool of application source code vulnerability diagnosis. It is quite curious to note that our system, with such robust generative AI models, a distributed modular architecture, and an interface that is readily comprehensible by the engineering teams, could lead to the modern era of DevSecOps by automatic security decision making and even beyond that.

6.2 Innovations and Achievements

The application of the Google Gemini AI model is credited to deep code inspection, and the presence of specific information regarding the line of code that has a deficiency, in addition to the correct identification of various overlapping security issues as stated by the OWASP guidelines. MasSecEval, in order to be able successfully lead both the security analyst and the developer through the remedial processes, makes sure that the work flow is hassle free even during the very first step of the repository ingestion process up to the dynamic trace validation services. By applying design patterns, which include Microservices, Publisher Subscriber, and strict Tenant Isolation, the system has become more maintainable, more scalable, and more responsive in addition, the component based structure has also made both the frontend clients and the backend workers to decouple their concerns and have a better telemetry data management.

6.3 User-Centered Features

The enhancement of the platform with such aspects as the use of a localized Retrieval Augmented Generation chat interface, a well analyzed automated compliance report, and the most secure way of mapping theoretical flaws to live OpenTelemetry traffic has been key in transforming MasSecEval to not only a static diagnosis tool but as one of the integral security monitoring platforms in software engineering. In additional to the fact that it automates the process of code auditing and lessening diagnostic fatigue levied on individual developers, MasSecEval can greatly

enhance the process of safe deployment particularly in the case where in the universe of an organization that does not have full-time security staff or where vulnerabilities alert of the organization are overwhelming and lacks basic importance.

6.4 Future Improvements

To break the identified limits and enhance the use of application and make the application performance and scalability better, a number of improvements are proposed:

6.4.1 Improved Local Model Integration

Continuous recalibration of the system to accommodate the localized open source large language models to be more accommodating of proprietary frameworks and internal coding standards will help in increasing organizational privacy. Localized model practices may be able to optimize modelling the analysis engine to new codebases to increase the accuracy rate of vulnerability detection concerning a variety of enterprise environments without heavily engaging external cloud APIs.

6.4.2 Active Fuzz Testing Capabilities

The current one heavily depends on passive telemetry surveillance to authenticate dynamic pathways. However, the active fuzzing functionality, such as the automated creation of malicious payloads to fuzz the API endpoints would be of great importance to thoroughly test the application in real-life situations. The functionality may be added by providing software penetration testing scripts that will run smoothly with the deployment.

6.4.3 Simplified Dashboard Interface

Developer centered design workshops, usability testing, will be made possible to refine the user interface. The triage workflow efficiency and reduction of the cognitive load when handling highly complex security alerts and trace spans will be minimized through continuous feedback provided by the software engineering professionals as a result of the iterative processes.

6.4.4 TimescaleDB Telemetry Integration

Research on integrating specialized time series databases can improve data retention and analytical metrics monitoring where a high amount of runtime events needs to be processed. Our focus on a specific TimescaleDB decentralised store can help decrease the bottlenecks in databases during peak traffic as well as create organizational confidence in the long term surveillance and integrity of the health data of their applications.

6.4.5 Expansion of Language Support

In later innovations, the inclusion of additional programming languages in addition to the already supported current stacks will attract more users in terms of Global enterprises. The presence of legacy system knowledgeable specialists and software architects of diverse backgrounds will offer accurate abstract syntax tree parsing and cultural modification of scanning elements to have the platform more friendly to older codebases.

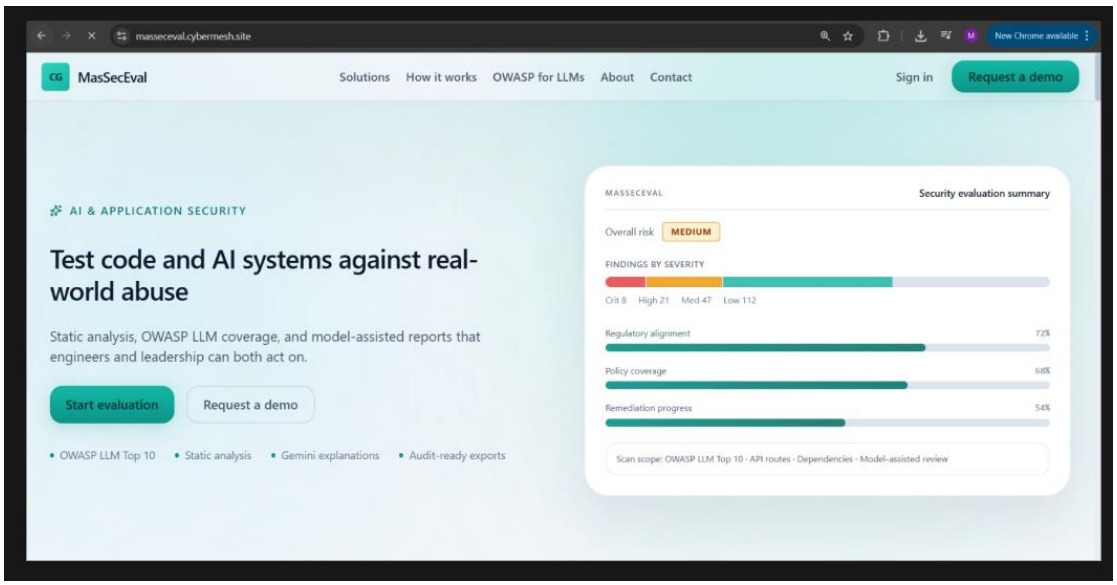
6.4.6 Real Time Collaboration Tools Integration

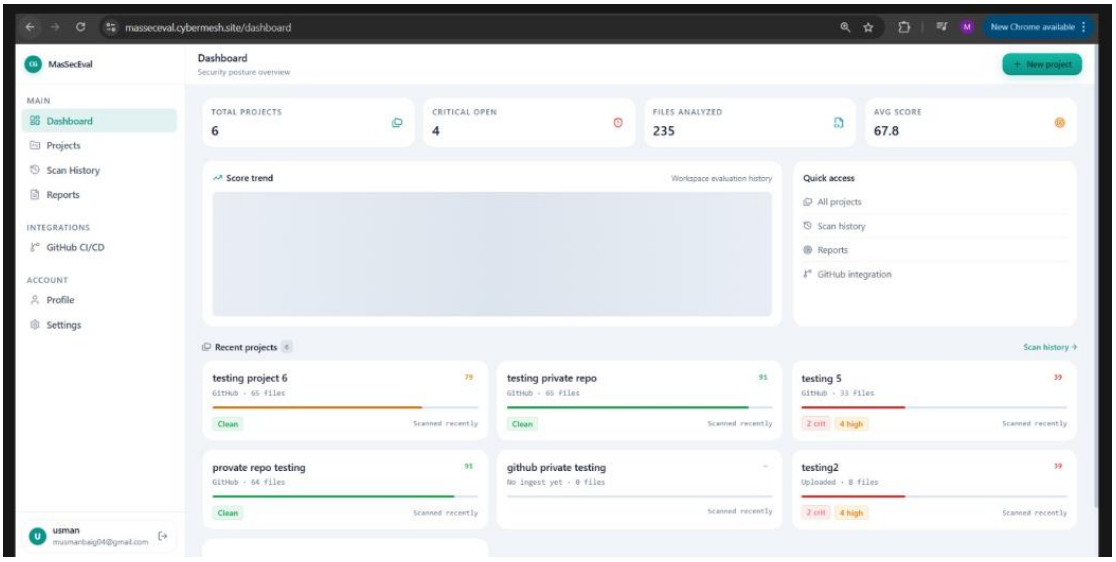
The collaboration technology such as Jira issue tracker and auto Slack notifications will be integrated to provide security professionals with quick response. It will promote combined patch planning and knowledge sharing which can enable the development and operations teams to coordinate effectively regardless of geographical location or departmental divisions.

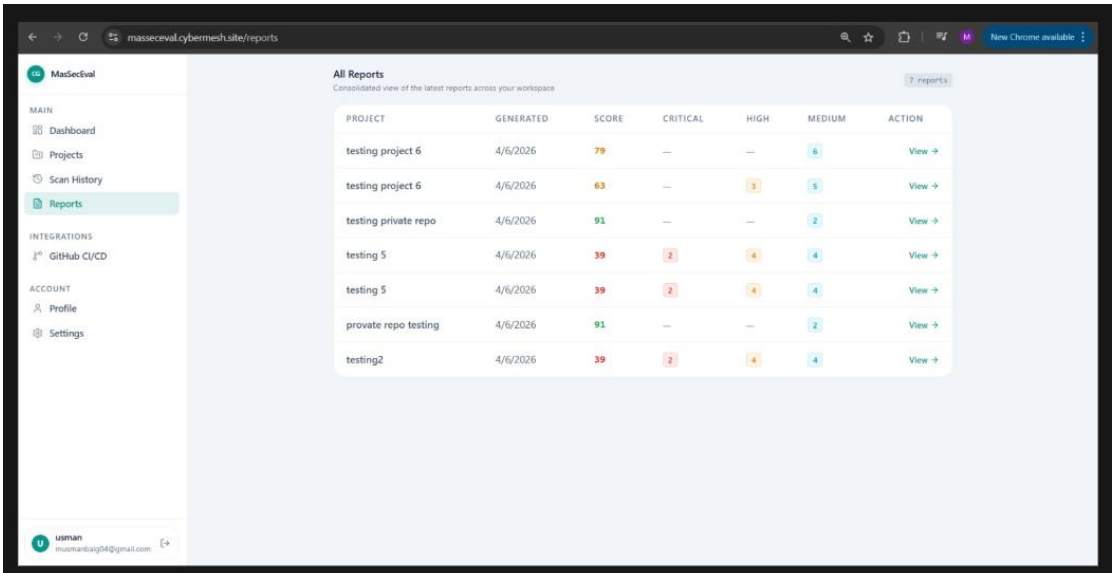
REFERENCES

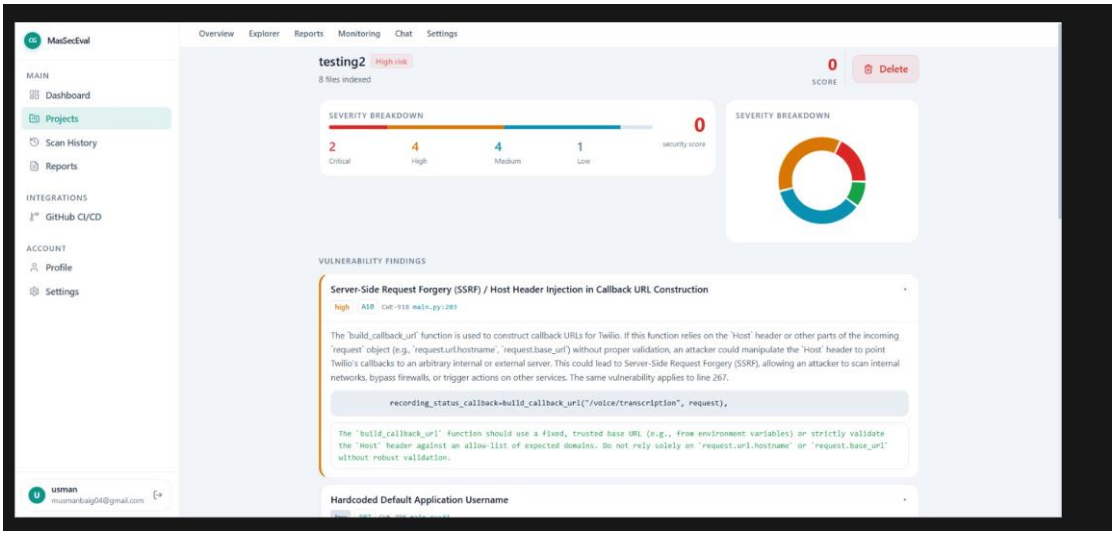
1. Pan Z, et al. Large Language Model for Vulnerability Detection and Repair: A Systematic Literature Review. arXiv preprint arXiv:2404.02525. 2024 Apr. Available: https://arxiv.org/abs/2404.02525.
2. Li X, et al. Towards Explainable Vulnerability Detection With Large Language Models. IEEE Transactions on Software Engineering. 2024. doi: 10.1109/TSE.2024.11146900.
3. Gemini Team, Google. Gemini: A Family of Highly Capable Multimodal Models. arXiv preprint arXiv:2312.11805. 2023 Dec. Available: https://arxiv.org/abs/2312.11805.
4. OpenTelemetry Contributors. OpenTelemetry: The open standard for telemetry. OpenTelemetry Documentation. 2024. Available: https://opentelemetry.io/
5. Palo Alto Networks. What Is Retrieval Augmented Generation (RAG)? An Overview. Palo Alto Networks Cyberpedia. 2024. Available: https://www.paloaltonetworks.com/cyberpedia/what-is-retrieval-augmented-generation
6. Cycode. What Is Static Application Security Testing (SAST)? Cycode Blog. 2024 Mar 26. Available: https://cycode.com/blog/what-is-static-application-security-testing-sast/
7. Amazon Web Services. Distributed Tracing AWS Distro for OpenTelemetry. AWS Documentation. 2024. Available: https://aws.amazon.com/otel/
8. OWASP Foundation. OWASP Top 10: The Ten Most Critical Web Application Security Risks. 2021. Available: https://owasp.org/Top10/
9. GitLab. GitLab 18.10 brings AI native triage and remediation. GitLab Blog. 2026 Mar 19. Available: https://about.gitlab.com/blog/gitlab-18-10-brings-ai-native-triage-and-remediation/
10. Amazon Web Services. What is RAG? Retrieval Augmented Generation AI Explained. AWS Cloud Fundamentals. 2024. Available: https://aws.amazon.com/what-is/retrieval-augmented-generation/

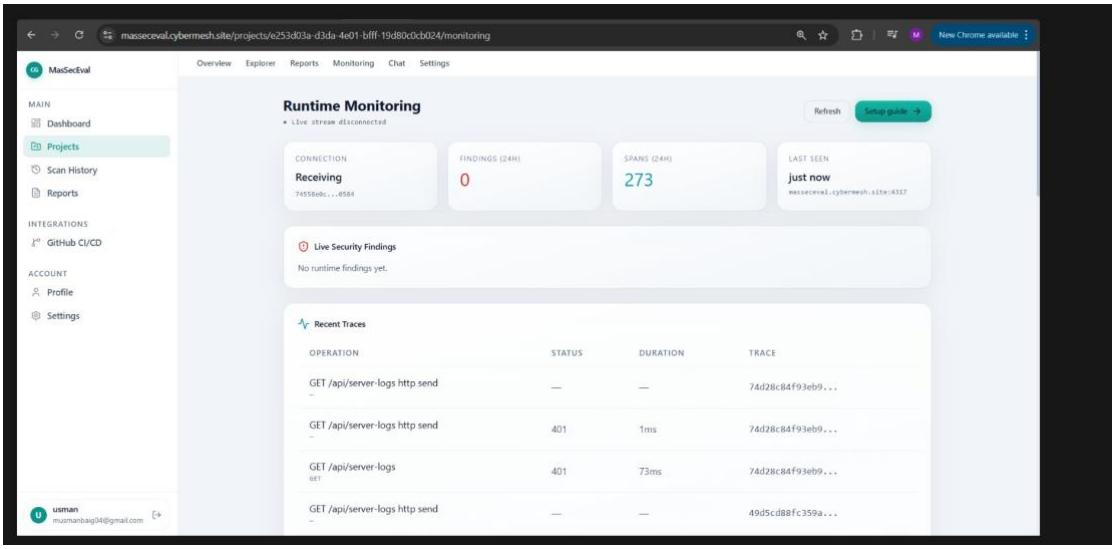
PLAGIARISM REPORT

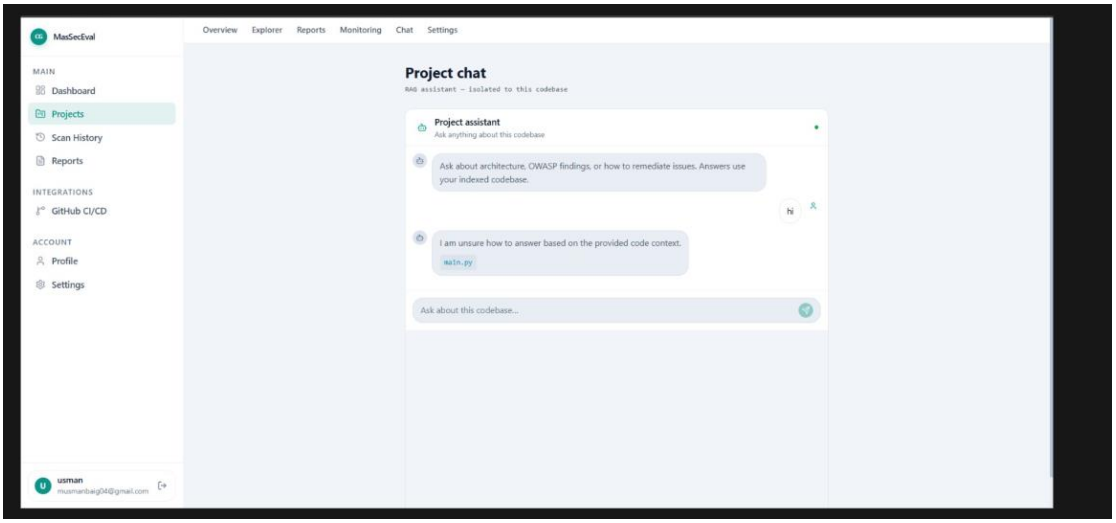


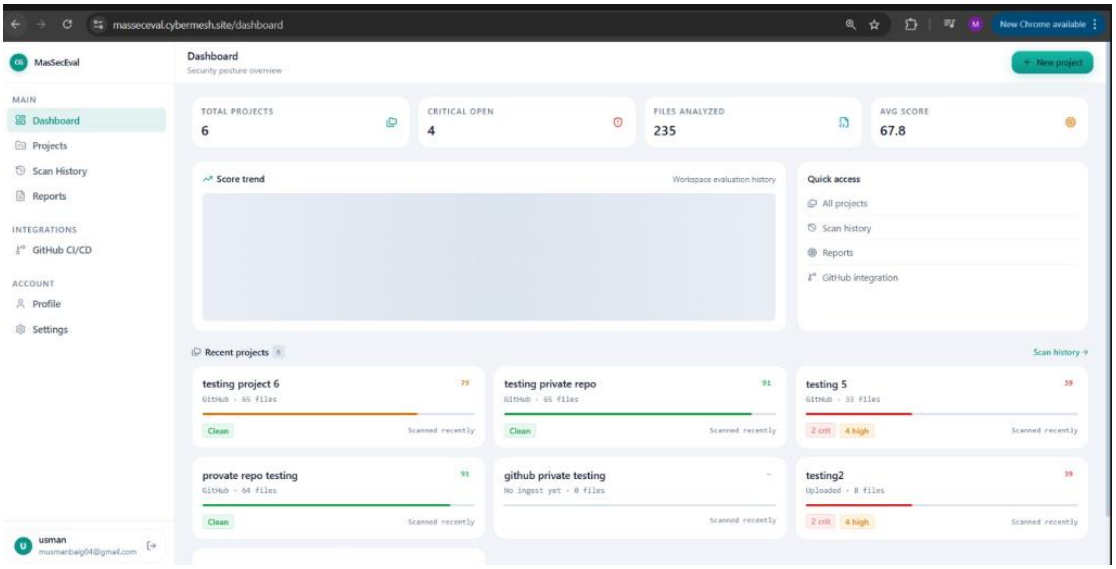

masseceval.cybermesh.site/dashboard
New Chrome available
MasSecEval
Dashboard
Security posture overview
+ New project
MAIN
Dashboard
Projects
Scan History
Reports
INTEGRATIONS
GitHub CI/CD
ACCOUNT
Profile
Settings
TOTAL PROJECTS
6
CRITICAL OPEN
4
FILES ANALYZED
235
AVG SCORE
67.8
Score trend
Workspace evaluation history
Quick access
All projects
Scan history
Reports
GitHub integration
Recent projects
Scan history →
testing project 6
GitHub · 65 files
79
Clean
Scanned recently
testing private repo
GitHub · 65 files
91
Clean
Scanned recently
testing 5
GitHub · 33 files
39
2 crit
4 high
Scanned recently
provate repo testing
GitHub · 64 files
91
Clean
Scanned recently
github private testing
No ingest yet · 0 files
Scanned recently
testing2
Uploaded · 8 files
39
2 crit
4 high
Scanned recently
usman
musmanbaig04@gmail.com

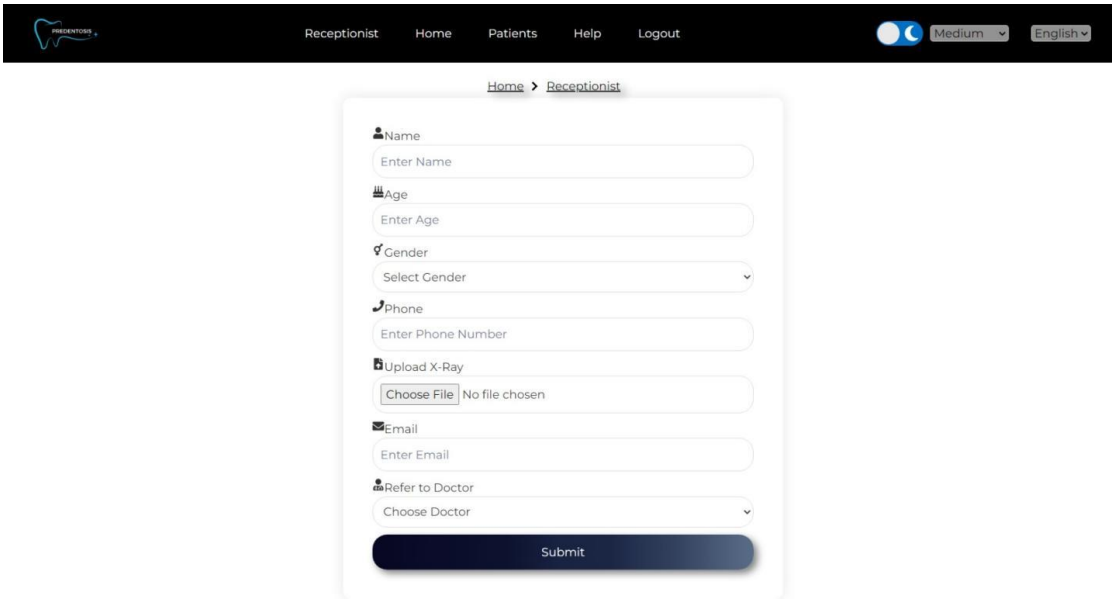

masseceval.cybermesh.site/reports
New Chrome available
MasSecEval
MAIN
Dashboard
Projects
Scan History
Reports
INTEGRATIONS
GitHub CI/CD
ACCOUNT
Profile
Settings
usman
musmanbaig04@gmail.com
All Reports
Consolidated view of the latest reports across your workspace
7 reports
PROJECT
GENERATED
SCORE
CRITICAL
HIGH
MEDIUM
ACTION
testing project 6
4/6/2026
79
—
—
6
View →
testing project 6
4/6/2026
63
—
3
5
View →
testing private repo
4/6/2026
91
—
—
2
View →
testing 5
4/6/2026
39
2
4
4
View →
testing 5
4/6/2026
39
2
4
4
View →
provate repo testing
4/6/2026
91
—
—
2
View →
testing2
4/6/2026
39
2
4
4
View →


MaxSecEval
MAIN
Dashboard
Projects
Scan History
Reports
INTEGRATIONS
GitHub CI/CD
ACCOUNT
Profile
Settings
usman
musmanbaig04@gmail.com
Overview
Explorer
Reports
Monitoring
Chat
Settings
testing2
High risk
8 files indexed
0
SCORE
Delete
SEVERITY BREAKDOWN
0
security score
2
Critical
4
High
4
Medium
1
Low
SEVERITY BREAKDOWN
VULNERABILITY FINDINGS
Server-Side Request Forgery (SSRF) / Host Header Injection in Callback URL Construction
high
A10
CWE-918
main.py:203
The `build_callback_url` function is used to construct callback URLs for Twilio. If this function relies on the `Host` header or other parts of the incoming `request` object (e.g., `request.url.hostname`, `request.base_url`) without proper validation, an attacker could manipulate the `Host` header to point Twilio's callbacks to an arbitrary internal or external server. This could lead to Server-Side Request Forgery (SSRF), allowing an attacker to scan internal networks, bypass firewalls, or trigger actions on other services. The same vulnerability applies to line 267.
recording_status_callback=build_callback_url("/voice/transcription", request),
The `build_callback_url` function should use a fixed, trusted base URL (e.g., from environment variables) or strictly validate the `Host` header against an allow-list of expected domains. Do not rely solely on `request.url.hostname` or `request.base_url` without robust validation.
Hardcoded Default Application Username


masseceval.cybermesh.site/projects/e253d03a-d3da-4e01-bfff-19d80c0cb024/monitoring
New Chrome available
MasSecEval
Overview
Explorer
Reports
Monitoring
Chat
Settings
MAIN
Dashboard
Projects
Scan History
Reports
INTEGRATIONS
GitHub CI/CD
ACCOUNT
Profile
Settings
usman
musmanbaig34@gmail.com
Runtime Monitoring
• Live stream disconnected
Refresh
Setup guide
CONNECTION
Receiving
FINDINGS (24H)
0
SPANS (24H)
273
LAST SEEN
just now
Live Security Findings
No runtime findings yet.
Recent Traces
OPERATION
STATUS
DURATION
TRACE
GET /api/server-logs http send
GET /api/server-logs http send
401
1ms
74d28c84f93eb9...
GET /api/server-logs
GET
401
73ms
74d28c84f93eb9...
GET /api/server-logs http send
49d5cd88fc359a...


MasSecEval
Overview
Explorer
Reports
Monitoring
Chat
Settings
MAIN
Dashboard
Projects
Scan History
Reports
INTEGRATIONS
GitHub CI/CD
ACCOUNT
Profile
Settings
usman
musmanbaig04@gmail.com
Project chat
RAG assistant — isolated to this codebase
Project assistant
Ask anything about this codebase
Ask about architecture, OWASP findings, or how to remediate issues. Answers use your indexed codebase.
hi
I am unsure how to answer based on the provided code context.
main.py
Ask about this codebase...


masseceval.cybermesh.site/dashboard
New Chrome available
MasSecEval
Dashboard
Security posture overview
+ New project
MAIN
Dashboard
Projects
Scan History
Reports
INTEGRATIONS
GitHub CI/CD
ACCOUNT
Profile
Settings
TOTAL PROJECTS
6
CRITICAL OPEN
4
FILES ANALYZED
235
AVG SCORE
67.8
Score trend
Workspace evaluation history
Quick access
All projects
Scan history
Reports
GitHub integration
Recent projects
Scan history →
testing project 6
GitHub · 65 files
79
Clean
Scanned recently
testing private repo
GitHub · 65 files
91
Clean
Scanned recently
testing 5
GitHub · 33 files
39
2 crit
4 high
Scanned recently
provate repo testing
GitHub · 64 files
91
Clean
Scanned recently
github private testing
No ingest yet · 0 files
–
Scanned recently
testing2
Uploaded · 8 files
39
2 crit
4 high
Scanned recently
usman
musmanbaig04@gmail.com


Receptionist
Home
Patients
Help
Logout
Medium
English
Home > Receptionist
Name
Enter Name
Age
Enter Age
Gender
Select Gender
Phone
Enter Phone Number
Upload X-Ray
Choose File
No file chosen
Email
Enter Email
Refer to Doctor
Choose Doctor
Submit

20% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Important Notice

The percentage shown reflects a probabilistic assessment and should be treated as an indicator, not a definitive conclusion. Results may vary depending on writing style, subject matter, and document structure. Always use this report as one of several points of reference rather than a sole determining factor.

Guidelines

This report analyzes long-form writing, such as essays, dissertations, and articles. Highlight colors indicate likelihood of AI detection flagging: cyan for high, light purple for moderate, unhighlighted for low.

Non-qualifying content (e.g. bullet points, annotated bibliographies, tables) is not analyzed and may cause the highlighted text to differ from the overall percentage shown. If the document contains non-selectable text, scanned images, or other formatting issues, the score may be inaccurate.

Details

Word Count: 18,337

Document Size: 2.24 MB

Score Interpretation

0% - 19%	Low likelihood of AI-generated content
20% - 50%	Moderate likelihood of AI-generated content
51% - 100%	High likelihood of AI-generated content

Highlight Legend

- High likelihood of being Flagged
- Moderate likelihood of being flagged
- Low likelihood of being flagged

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **75** Not Cited or Quoted **6%**
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations **0%**
  Matches that are still very similar to source material
- **0** Missing Citation **0%**
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted **0%**
  Matches with in-text citation present, but no quotation marks

Top Sources

- 4% Internet sources
- 1% Publications
- 4% Submitted works (Student Papers)

Match Groups

- **75** Not Cited or Quoted 6%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
1% Publications
4% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	repositories.nust.edu.pk <1%
2	Student papers	Higher Education Commission Pakistan <1%
3	Internet	www.coursehero.com <1%
4	Internet	eprints.utar.edu.my <1%
5	Student papers	The University of the South Pacific <1%
6	Student papers	Islington College,Nepal <1%
7	Student papers	Colorado Technical University Online <1%
8	Internet	cscgp.miuegypt.edu.eg <1%
9	Student papers	University of South Florida <1%
10	Student papers	Blue Mountain Hotel School <1%

11	Internet	
vtechworks.lib.vt.edu		<1%
12	Internet	
brandlab.com.au		<1%
13	Student papers	
lahore garrison LGU		<1%
14	Student papers	
Swinburne University of Technology		<1%
15	Student papers	
Universitat Oberta de Catalunya		<1%
16	Internet	
about.gitlab.com		<1%
17	Publication	
M. Shafiur Rahman. "Handbook of Food Preservation", CRC Press, 2019		<1%
18	Internet	
ir.aiktclibrary.org:8080		<1%
19	Publication	
Alim Hardiansyah. "Designing Android Based Augmented Reality Location-Based ...		<1%
20	Publication	
H.L. Gururaj, Francesco Flammini, S. Srividhya, M.L. Chayadevi, Sheba Selvam. "Co...		<1%
21	Student papers	
KAZGUU University		<1%
22	Student papers	
Nelson Marlborough Institute of Technology		<1%
23	Student papers	
University of North Texas		<1%
24	Student papers	
Visvesvaraya Technological University		<1%

25 Student papers

Edith Cowan University <1%

26 Publication

Hsu, Ke-Jou. "System Support for Fine-Grained Resource Management in Mobile E... <1%

27 Publication

Kesara Wimal. "A Truly Decentralized Consensus Protocol that Eliminates Tenden... <1%

28 Student papers

Universidad Internacional de la Rioja <1%

29 Internet

ruor.uottawa.ca <1%

30 Internet

www.contractsfinder.service.gov.uk <1%

31 Internet

www.nanda-lab.ca <1%

32 Internet

www.paloaltonetworks.com <1%

33 Internet

harvest.usask.ca <1%

34 Student papers

University of the West Indies <1%

35 Internet

scholar.ppu.edu <1%

36 Internet

www.mcours.net <1%

37 Internet

www.slideshare.net <1%

38 Internet

123dok.com <1%

39 Publication

Ghimire, Prashnna. "Framework for Integrating Industry Knowledge into a Large ... <1%

40 Publication

Mark S. Merkow. "Secure, Resilient, and Agile Software Development", CRC Press, ... <1%

41 Internet

eprints.usm.my <1%

42 Internet

proceedings.neurips.cc <1%

43 Internet

research-information.bris.ac.uk <1%

44 Publication

Baptista, Tiago João Fernandes. "Fast Scan, An Improved Approach Using Machin... <1%

45 Publication

James Ransome, Anmol Misra. "Core Software Security - Security at the Source", A... <1%